

Production Hardening for Data & ML Systems

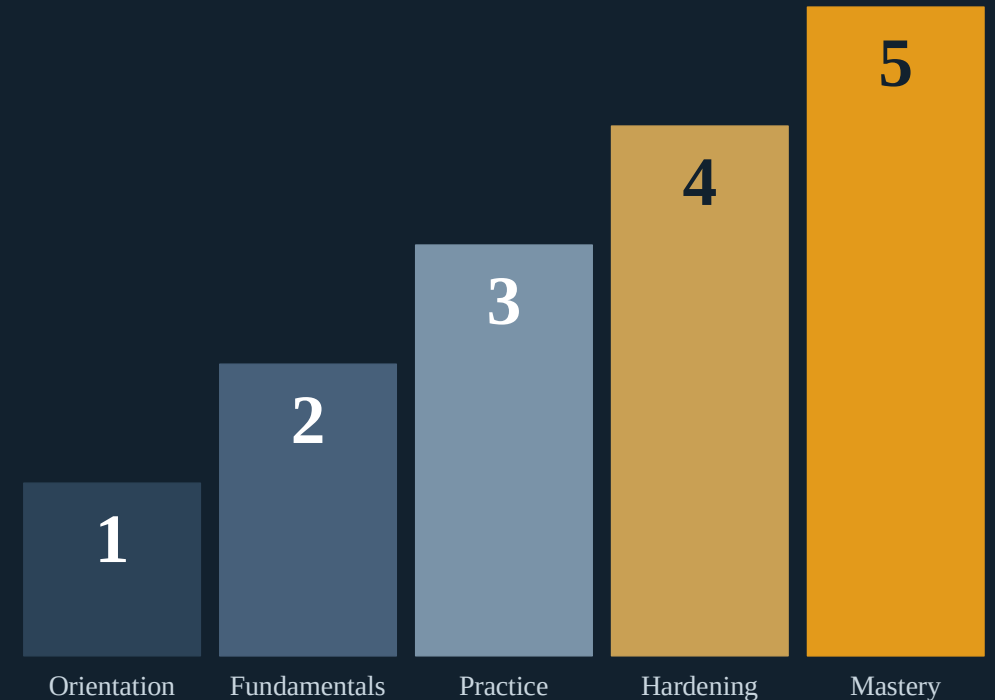
CI/CD for data · Terraform · Monitoring and alerting · Incident response · Cost review · Idempotency · Backfills

A field tutorial in nine parts. Running example: an illustrative retail demand-forecasting platform.

HOW TO USE

Read top to bottom once. The footer gauge rises as the material gets harder.

Depth staircase: every slide carries a gauge in its footer



Roadmap: nine parts, five depth levels

Each part ends with a gate checklist you can lift straight into a production readiness review.

Part	Topic	Slides	Depth	By the end you can...
1	Framing the problem	1–14	1	Name the failure modes and score your platform against a maturity ladder
2	Idempotency	15–34	1–2	Make any job safe to run twice, and prove it with an automated test
3	Backfills	35–54	2–3	Reprocess history without harming production or trust in the data
4	CI/CD for data and ML	55–76	3	Ship pipeline, contract and model changes behind automated gates
5	Terraform	77–96	3–4	Change the platform with reviewed, tested, drift-checked code
6	Monitoring and alerting	97–120	4	Detect problems by SLO burn instead of by noise
7	Incident response	121–134	4–5	Contain, communicate and learn from data incidents
8	Cost review	135–144	5	Run a monthly review that ties spend to unit economics
9	Capstone	145–150	5	Run a readiness review and plan the first 90 days

THE PREMISE

Assume every job will run twice, every partition will be rebuilt, and every alert will fire at 03:00.

Parts 2–3 · Re-runs and rebuilds change nothing they should not

Part 4 · Every change ships through automated gates

Part 5 · Infrastructure is reviewed code, not console clicks

Part 6 · Alerts are rare, actionable and tied to SLOs

Part 7 · Incidents are contained fast and learned from

Part 8 · Spend maps to a unit cost that someone owns

Prototype versus hardened

The same pipeline, before and after hardening.

Prototype		Hardened
Appends on every run, so a retry doubles the rows	→	Writes by key or partition, so a retry changes nothing
Uses now() and "latest", so results depend on when it ran	→	Takes a logical date, so results depend only on inputs
Deployed by hand from a laptop or notebook	→	Merged, tested and released by a pipeline from git
Infrastructure clicked together in a console	→	Infrastructure declared, reviewed and drift-checked
Failure discovered when a stakeholder complains	→	An SLO alert catches failure before anyone notices
Cost discovered on the invoice	→	Cost tagged, budgeted and reviewed every month

DEFINITION

Production hardening: the engineering work that keeps a working system correct, recoverable and affordable under retries, bad inputs, bad deploys and growth.

Eight ways data systems actually fail

Every control in this deck exists to stop, shrink or expose one of these.

LATE**1****Late data**

- Upstream lands after the consumer's deadline.
- Control: freshness SLO and deadline alert (Part 6).

DUPLICATE**2****Duplicate data**

- A retry or replay appends the same rows twice.
- Control: keyed merge or partition overwrite (Part 2).

SILENT**3****Wrong data, silently**

- The job succeeds and the numbers are off.
- Control: contracts, quality tests, drift monitors (Parts 4, 6).

MISSING**4****Missing data**

- A partition or key range never arrived.
- Control: completeness checks, ledgered backfills (Parts 3, 6).

SCHEMA**5****Schema drift**

- Upstream renames or retypes a column.
- Control: data contracts and compatibility checks in CI (Part 4).

DEPLOY**6****Bad deploy**

- A merged change breaks or corrupts output.
- Control: slim CI, data diff, fast rollback (Part 4).

INFRA**7****Infrastructure failure or drift**

- A console edit, expired credential or deleted bucket.
- Control: Terraform and drift detection (Part 5).

COST**8****Cost blowout**

- A runaway backfill, forgotten cluster or full scans.
- Control: tags, budgets, anomaly alerts (Part 8).

RULE

Incident response (Part 7) is the safety net under all eight: contain, communicate, learn.

Failures are not equal: loud versus silent

Silent failures with large impact are the ones that end careers, so they get hardened first.

Impact: small ↑ large

Silent and large

- A join fan-out inflates revenue by 8%
- Retries duplicate a day of orders
- A model scores from stale features

Loud and large

- The warehouse is down at 06:00
- The pipeline fails and the dashboard is empty
- A credential expires for every job

Silent and small

- A rare null in a minor column
- Slow drift in a low-value feature
- An unused table keeps growing

Loud and small

- A retryable task failure
- A transient API timeout
- A single flaky test

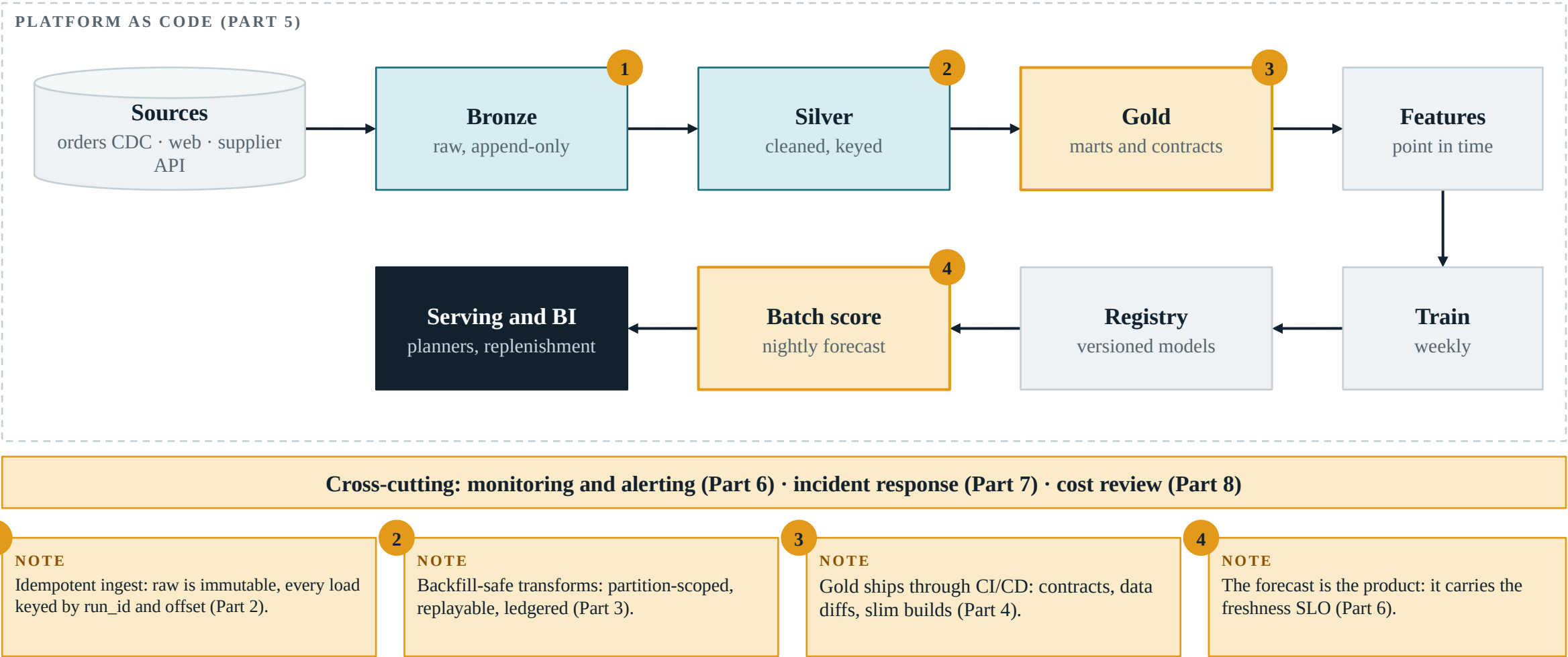
Detectability: silent → loud

PRACTICE

Prioritise by impact times undetectability. Loud failures get fixed because people shout; silent ones need engineered detection.

Running example: the Meridian demand-forecast platform

A composite, illustrative system. Numbered badges show where each part of this tutorial hardens it.



The promises: service levels for the running example

Everything later (alerts, incidents, cost) hangs off these numbers.

Consumer	Promise (SLO)	Indicator (SLI)	Window
Merchandising planners	gold.forecast lands before 06:00 on 99% of days	Landing time versus 06:00 Europe/London	30 days
Finance	Orders in gold match the source of record within 0.1%	Daily reconciliation of counts and totals	30 days
Store replenishment	Forecast covers every active SKU-store pair	Share of active pairs with a forecast row	Daily
Data science	Features are never older than 26 hours at scoring	Maximum feature timestamp lag at score start	30 days
Platform owners	Spend stays within budget at a target unit cost	Cost per 1,000 forecasts from tagged billing data	Calendar month

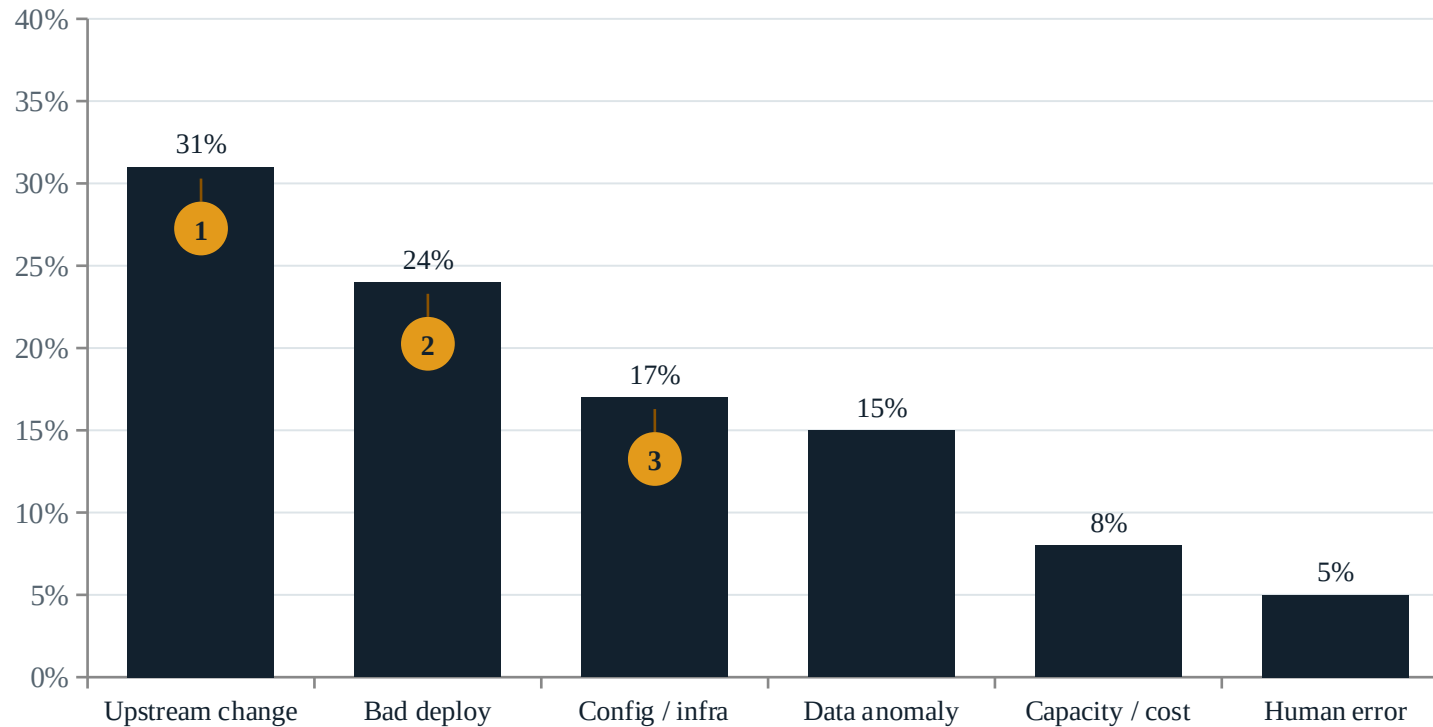
RULE

If you cannot state the promise as a number, you cannot alert on it, review it or price it.

Where incidents come from

Most incidents start upstream of the code you wrote.

Figure 1. Share of incidents by root cause, % (illustrative composite; calibrate with your own postmortems)



1

NOTE

The largest bucket. Contracts (Part 4) and freshness monitors (Part 6) target it.

2

PRACTICE

Gated deploys and data diffs shrink this one (Part 4).

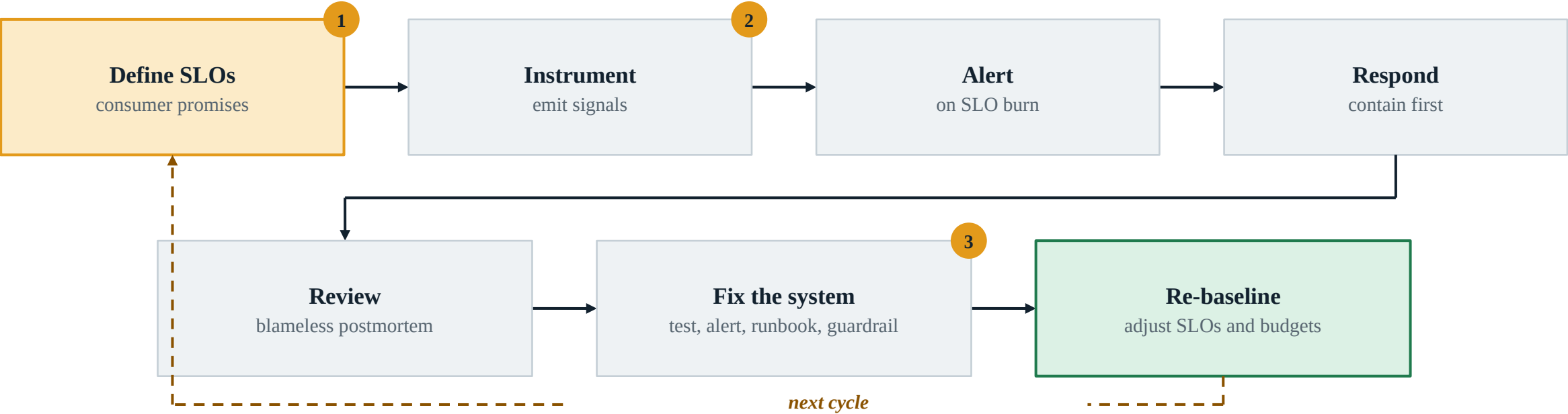
3

PRACTICE

Terraform and drift checks turn config errors into reviewed diffs (Part 5).

The hardening loop

Hardening is a cycle, not a project: every incident should change something.



1

NOTE
Start from consumer promises, not from tool defaults.

2

PRACTICE
Emit run_id, partition and outcome from every job.

3

DISCIPLINE
An incident with no system change will repeat.

Six design principles

Referenced by number throughout the deck.

PRINCIPLE 1

Idempotent by default

- Retries, replays and backfills must be safe.
- Prefer partition overwrite or keyed merge over append.

PRINCIPLE 2

Immutable inputs, versioned outputs

- Never edit raw data in place.
- Every output records run_id, code version and input snapshot.

PRINCIPLE 3

Fail loudly, publish atomically

- Validate before consumers can see the data.
- Write-audit-publish beats fix-it-afterwards.

PRINCIPLE 4

Everything as code

- Pipelines, infrastructure, alerts and dashboards live in git.
- Review, test and roll back like software.

PRINCIPLE 5

Observe outcomes, not just tasks

- A green DAG with wrong data is a failure.
- Measure freshness, volume, validity and drift.

PRINCIPLE 6

Cost is a design constraint

- Tag, budget and estimate before you run.
- Unit cost is a first-class metric.

A maturity ladder you can score yourself on

Score each column separately. The gaps between columns are your roadmap.

Level	Idempotency and backfills	CI/CD and infrastructure	Monitoring and alerting	Incidents and cost
0 · Ad hoc	Appends; manual reruns	Notebooks; console clicks	Users report problems	Heroics; invoice surprises
1 · Repeatable	Some jobs merge by key	Git and basic CI	Task-failure emails	Informal channel; monthly bill view
2 · Managed	Run-twice tests; ledgered backfills	Gated deploys; Terraform for the core	Freshness and volume SLOs	On-call and runbooks; tagged spend
3 · Measured	Shadow tables and diffs on every backfill	Slim CI, data diffs, drift checks	Burn-rate alerts; lineage-aware routing	Tracked postmortems; unit costs
4 · Optimised	Automated safe replays; spot compute	Policy as code; progressive delivery	Alerts tuned by review; SLO reviews	Game days; cost SLOs in reviews

NOTE

Most teams sit at level 1–2 in some columns and 3 in others. Aim to lift the lowest column first.

Five depth levels: what changes as you go deeper

The footer gauge tells you where you are.

1	Depth 1 · Orientation	Vocabulary, failure modes and the first patterns. Slides 1–22.	OUTCOME <i>You leave with: shared vocabulary and the run-twice test</i>
2	Depth 2 · Fundamentals	Patterns in code, with the reasoning behind each choice. Slides 23–52.	OUTCOME <i>You leave with: idempotent jobs and a backfill lifecycle</i>
3	Depth 3 · Practice	Pipelines, environments and infrastructure wired end to end. Slides 53–92.	OUTCOME <i>You leave with: CI/CD and Terraform workflows to adapt</i>
4	Depth 4 · Hardening	SLO maths, alert design, containment and failure-mode analysis. Slides 93–128.	OUTCOME <i>You leave with: burn-rate alerts and an incident runbook</i>
5	Depth 5 · Mastery	Trade-offs, organisational loops and running the whole system. Slides 129–150.	OUTCOME <i>You leave with: a review cadence and a 90-day plan</i>

READING ADVICE

Skim depth 1–2 if you already run production pipelines. Read depth 4–5 slowly.

Conventions and prerequisites

How to read the diagrams, code and figures.

EXAMPLE

Running example

- Meridian Retail is a composite.
- All names, volumes, prices and incident details are illustrative.

STACK

Tooling

- SQL and Python, Spark-style engines, an Airflow-style orchestrator, dbt-style models.
- Terraform examples use AWS; the ideas map to other clouds.

NOTATION

Reading the visuals

- Amber numbered badges link an element to its callout.
- Red = pitfall, green = practice, teal = definition.

ASSUMED

Prerequisites

- Comfortable with SQL, Python and git.
- Basic cloud concepts: storage, IAM, networking.

CAVEAT

Code samples are teaching-sized. Check syntax and behaviour against your own tool versions before adopting them.

02

Idempotency

Running a job twice must leave the world exactly as running it once.

SLIDES 15-34 · DEPTH 1-2

IN THIS PART

- Definition and the run-twice test
- Anatomy of non-idempotent jobs
- Overwrite, merge and write-audit-publish
- Time, keys, late data and deletes
- Side effects and streaming
- Orchestrators, ML and a run ledger
- Testing idempotency in CI

Idempotency, defined precisely

One idea, three levels of precision.

MATHEMATICS

$$f(f(x)) = f(x)$$

- Applying the operation again changes nothing.
- Absolute value and set-union are idempotent; "add 1" is not.

PIPELINES

Same inputs, same end state

- Run a job once or five times against the same inputs and the target ends up identical.
- Intermediate states may differ; the final state may not.

DELIVERY

At-least-once plus idempotent = effectively once

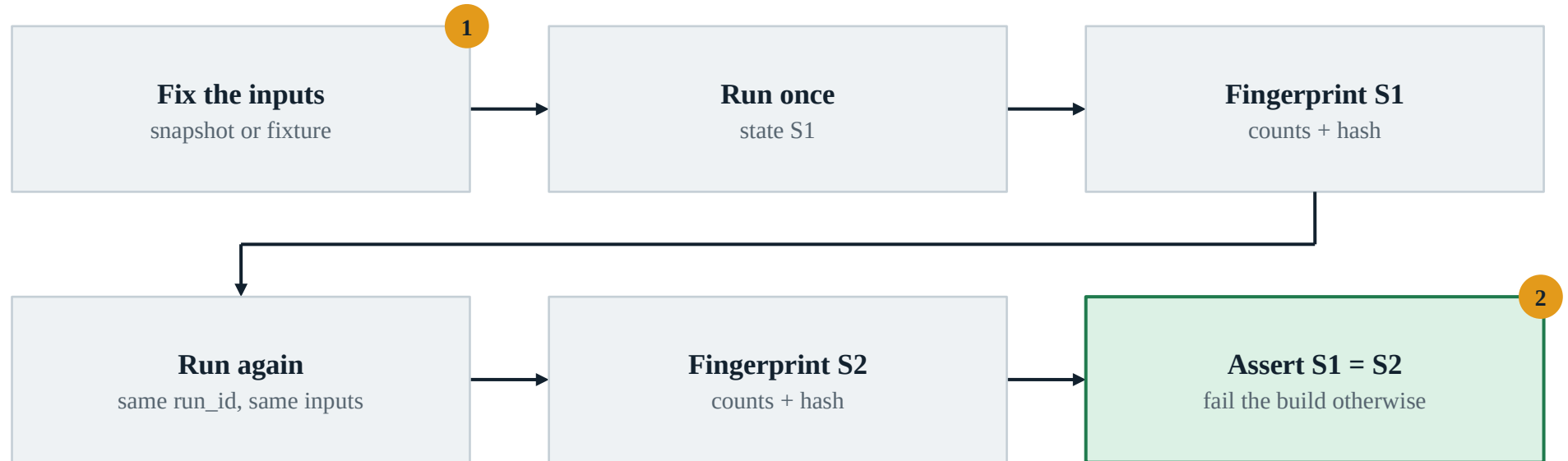
- Exactly-once delivery is rarely available end to end.
- Accept duplicates in transit and make the sink absorb them.

RULE

Idempotency is a property of the whole write path: inputs, transform, sink and any side effect.

The run-twice test

The simplest and most valuable automated check in this deck.



1

NOTE

Use a frozen input so the only variable is the job itself.

2

PRACTICE

Compare row counts and an order-independent hash, not just "no error".

EXTEND IT

Add variants: kill the job midway, then rerun; run two copies concurrently.

Anatomy of a non-idempotent job

Four defects hide in six lines of innocent-looking SQL.

```
-- runs nightly at 02:00
INSERT INTO gold.daily_orders
SELECT uuid() AS row_id,
       o.*,
       current_timestamp() AS loaded_at
FROM   silver.orders o
WHERE  o.order_ts >= now() - INTERVAL 1 DAY
LIMIT 100000;
```

SQL

1

APPEND

Blind INSERT appends. A retry doubles the day.

1

2

2

RANDOM KEY

uuid() differs per run, so no two runs produce the same key.

3

4

3

WALL CLOCK

now() slides the window: rerun at 09:00 and the input set is different.

4

UNORDERED

LIMIT without ORDER BY picks arbitrary rows.

RULE

Each defect has a standard fix: overwrite or merge, deterministic keys, a logical date, and a total order.

Sources of non-determinism

A checklist to run against any job you review.

Source	Symptom	Fix
Blind append	Retries duplicate rows	Overwrite the partition or MERGE on a key
now(), "latest"	Results depend on when the job ran	Pass a logical date or snapshot id as an input
Auto-increment and uuid keys	Same record gets a new id each run	Derive keys from business keys with a hash
Unordered LIMIT, first(), sampling	Different rows on each run	Total order, or a seeded and stable sample
External side effects	Emails, webhooks and payments repeat	Idempotency keys and an outbox
Reading mutable state	Upstream table changed between runs	Read a pinned snapshot or versioned partition
Time zones and DST	Daily windows shift twice a year	Compute in UTC; convert only at the edge
Floating-point order	Sums differ in the last digits	Fixed decimal types or rounding before compare

Pattern 1: overwrite the partition

Best for time-partitioned facts where a run owns a whole partition.

```
spark.conf.set(
    "spark.sql.sources.partitionOverwriteMode", "dynamic")

day = "2026-09-19" # logical date, passed in

df = (spark.table("silver.orders")
      .where(f"order_date = '{day}'"))

(df.write
 .mode("overwrite")
 .insertInto("gold.daily_orders"))
```

PySpark

1

CONFIG

Dynamic mode replaces only partitions present in the output. Static mode can wipe the whole table.

1

2

2

LINE 4

The date is an argument, never computed inside the job.

3

3

OVERWRITE

A rerun replaces exactly one partition, so the result is identical.

NOTE

Table formats offer a sharper tool: Delta `replaceWhere` or Iceberg `partition overwrite` replace exactly the predicate you name, atomically.

Pattern 2: MERGE on a key

Best for entity tables where records update over time.

SQL

```
MERGE INTO gold.orders AS t
USING (
  SELECT * FROM (
    SELECT *, row_number() OVER (
      PARTITION BY order_id
      ORDER BY updated_at DESC) AS rn
    FROM staging.orders_delta) x
  WHERE rn = 1) AS s
ON t.order_id = s.order_id
WHEN MATCHED AND s.updated_at > t.updated_at
THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

1

DEDUPE

Deduplicate the source first. Duplicate source keys make MERGE ambiguous or error.

1

2

KEY

The match key must be unique in the target too.

2

3

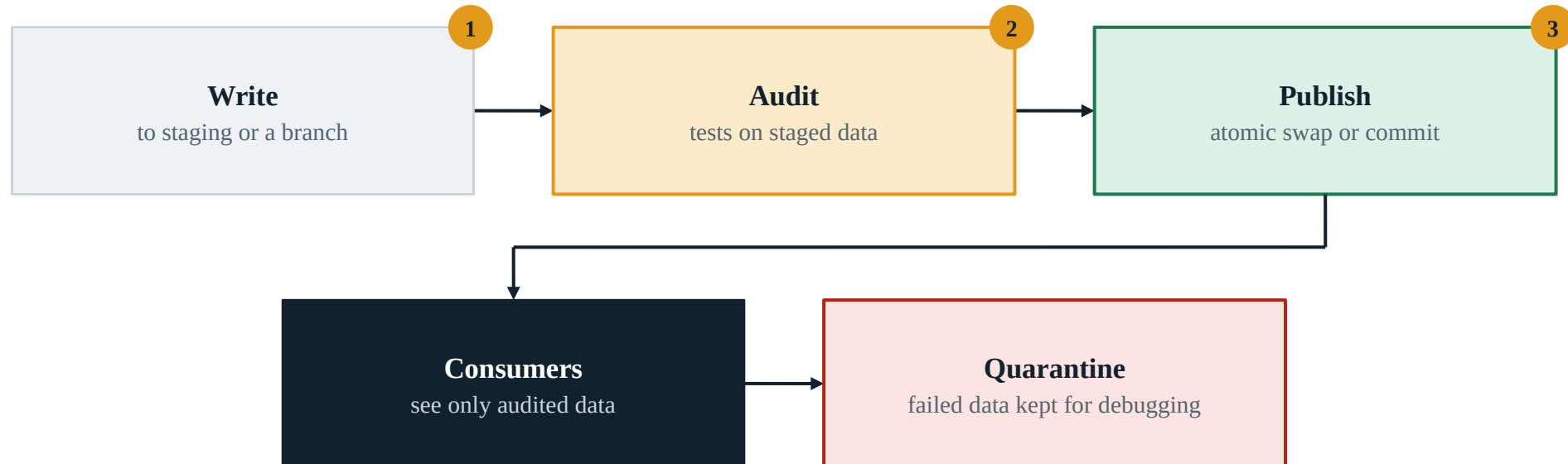
3

GUARD

The ordering guard makes late or replayed records harmless.

Pattern 3: write-audit-publish

Land data somewhere private, prove it is good, then expose it atomically.



1

NOTE

Consumers never read staging, so partial writes are invisible.

2

PRACTICE

Counts, keys, null rates and reconciliation run here, before anyone sees the data.

3

NOTE

Publishing is one metadata operation, so it either happens or does not.

Choosing a write pattern

Depth 2 begins: choose deliberately, not by habit.

Pattern	Best for	Atomic?	Main pitfall
Partition overwrite	Facts keyed by date; whole-partition ownership	Per partition	Static overwrite mode clears the whole table
MERGE / upsert	Mutable entities; CDC applies	Per statement	Duplicate source keys; missing ordering guard
Delete + insert in a transaction	Engines without MERGE	Yes, if transactional	Non-transactional stores expose the gap
Stage and swap	Large rebuilds; dimension tables	Yes	Needs an atomic pointer or rename
Write-audit-publish	Anything consumers depend on	Yes	Extra storage and one more moving part
Append with dedupe on read	Immutable event logs	Yes	Readers must all dedupe correctly

PRACTICE

Default: overwrite by partition for facts, MERGE for entities, and wrap either in write-audit-publish when consumers are critical.

Time is an input, not the weather

Logical time makes a rerun reproduce history instead of rewriting it.

Wall-clock thinking

WHERE ts > now() - 1 day

Yesterday's job, rerun today, reads today's data

Backfill needs code edits to change dates

Daylight-saving days have 23 or 25 hours in local time

Logical-time thinking

WHERE ts >= :window_start AND ts < :window_end

The window comes from the schedule slot, not the clock

Backfill is the same job run with different arguments

Windows are computed in UTC and half-open [start, end)

RULE

Half-open windows [start, end) mean adjacent runs never overlap and never leave gaps.

Deterministic keys instead of random ones

The same business record must get the same identity on every run.

Python

```
import hashlib, uuid

NS = uuid.UUID("6f1c2b7e-0a4d-4c55-9a51-3b1f8f0c2d11")

def row_key(*parts: str) -> str:
    raw = "|".join(p.strip().lower() for p in parts)
    return hashlib.sha256(raw.encode()).hexdigest()

def entity_id(kind: str, natural_key: str) -> uuid.UUID:
    return uuid.uuid5(NS, f"{kind}:{natural_key}")

# key = row_key(order_id, line_no, sku)
```

1

NORMALISE

Normalise before hashing, or "A1" and "a1 " become different keys.

1

2

2

LINE 7

A content hash is stable across runs, machines and languages.

3

3

UUID5 vs UUID4

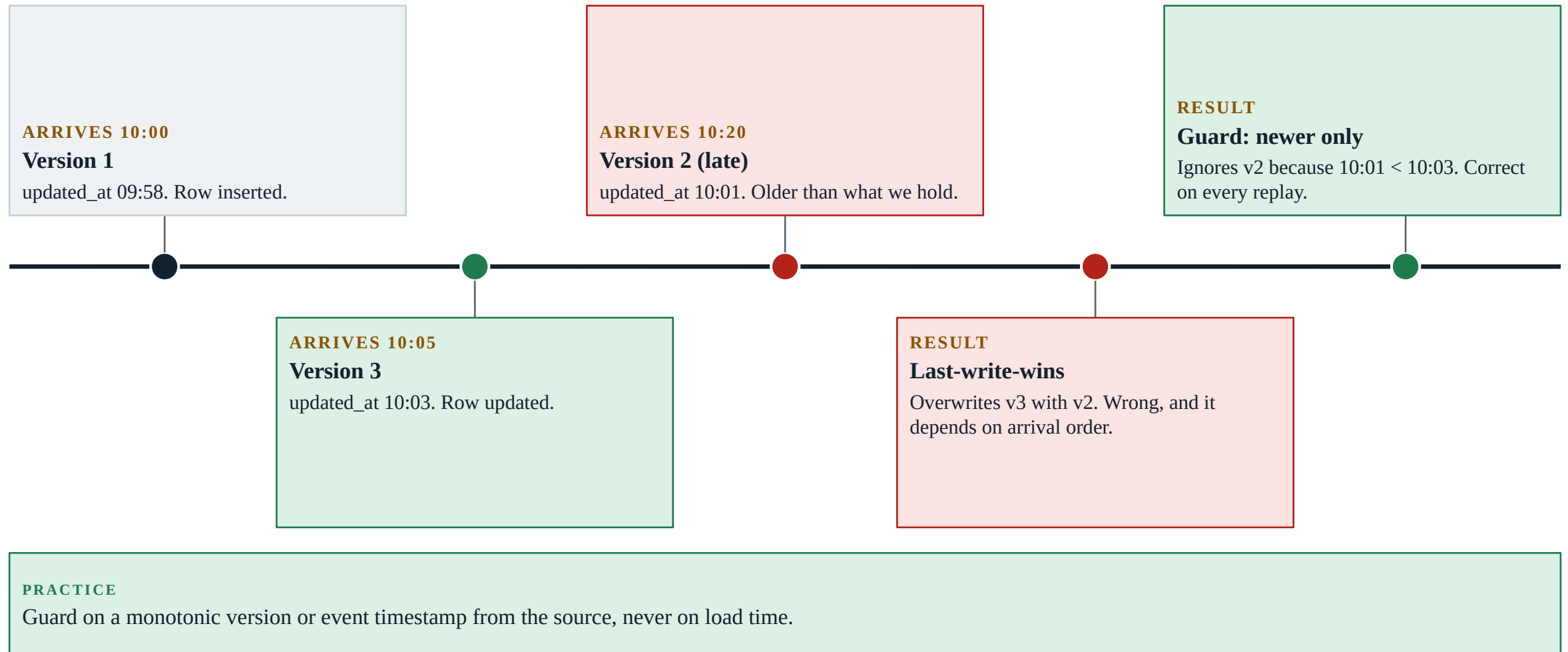
uuid5 is deterministic; uuid4 is random and breaks reruns.

PITFALL

Hash keys are for identity, not secrecy. If an attacker can guess the inputs, they can recompute the key.

Late and out-of-order data

Arrival order is not event order. Ordering guards make replays safe.



Deletes are data too

A pipeline that only knows how to add cannot represent removal.

APPROACH A

Hard delete

- Remove the row downstream.
- Simple, but a replay of the old insert can resurrect it.

APPROACH B

Soft delete

- Keep the row with `is_deleted` and `deleted_at`.
- Replays converge; readers must filter.

APPROACH C

Tombstones

- Emit an explicit delete event with a version.
- The version guard makes it order-safe.

APPROACH D

Full-partition rebuild

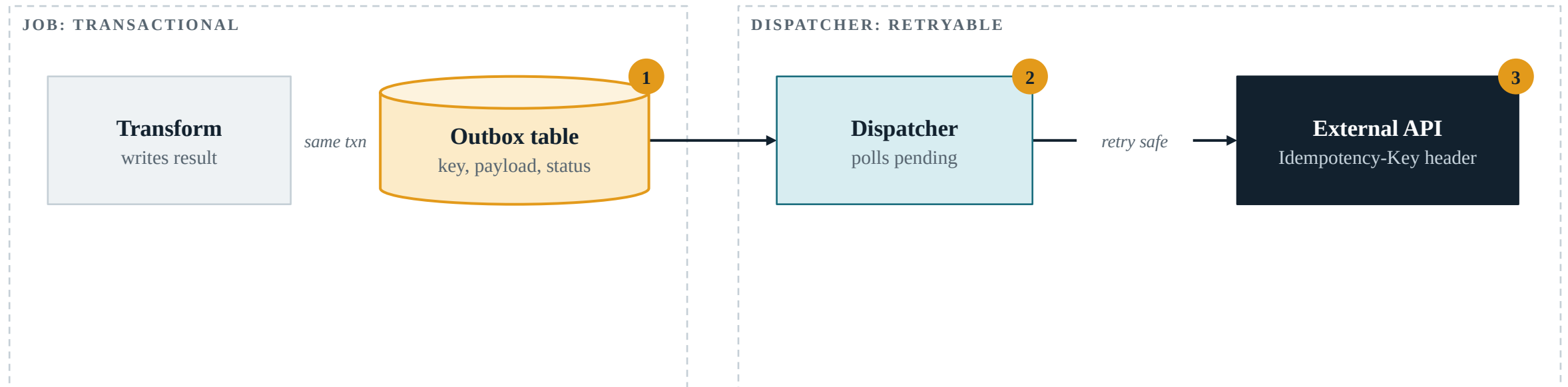
- Rebuild the whole partition from source.
- Deletes fall out naturally, at a higher cost.

COMPLIANCE

Erasure requests (for example under UK GDPR) need true removal, including from backups and derived tables. Plan for it separately from soft deletes.

Idempotent side effects: the outbox pattern

Emails, webhooks and payments must not repeat when the job does.



1

NOTE

The message is written in the same transaction as the result. Both exist or neither does. The message key is deterministic, for example a hash of run_id and entity id.

2

PRACTICE

The dispatcher can crash and retry freely. It marks a row sent only after the API acknowledges, so at worst a message is sent twice.

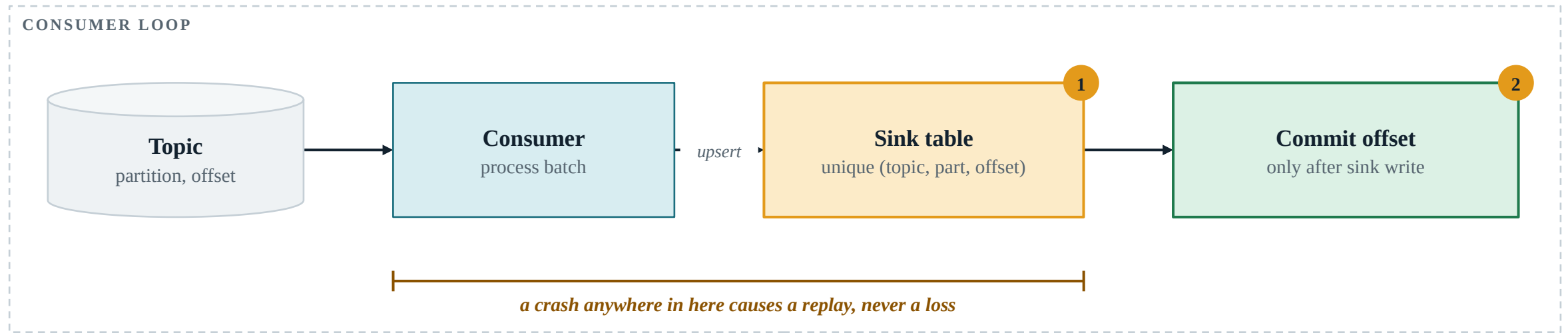
3

CHECK THE API

A duplicate is harmless only if the receiver honours the idempotency key. Confirm it does, and how long it remembers keys.

Effectively once in streaming

At-least-once transport plus an idempotent sink.



1

PRACTICE

A uniqueness constraint or MERGE on (topic, partition, offset), or on a business key, discards replays. The sink is where exactly-once is manufactured.

2

ORDER MATTERS

Commit the offset before the write and a crash loses data. Commit after and a crash duplicates it. Choose duplicates and dedupe.

TRANSACTIONS

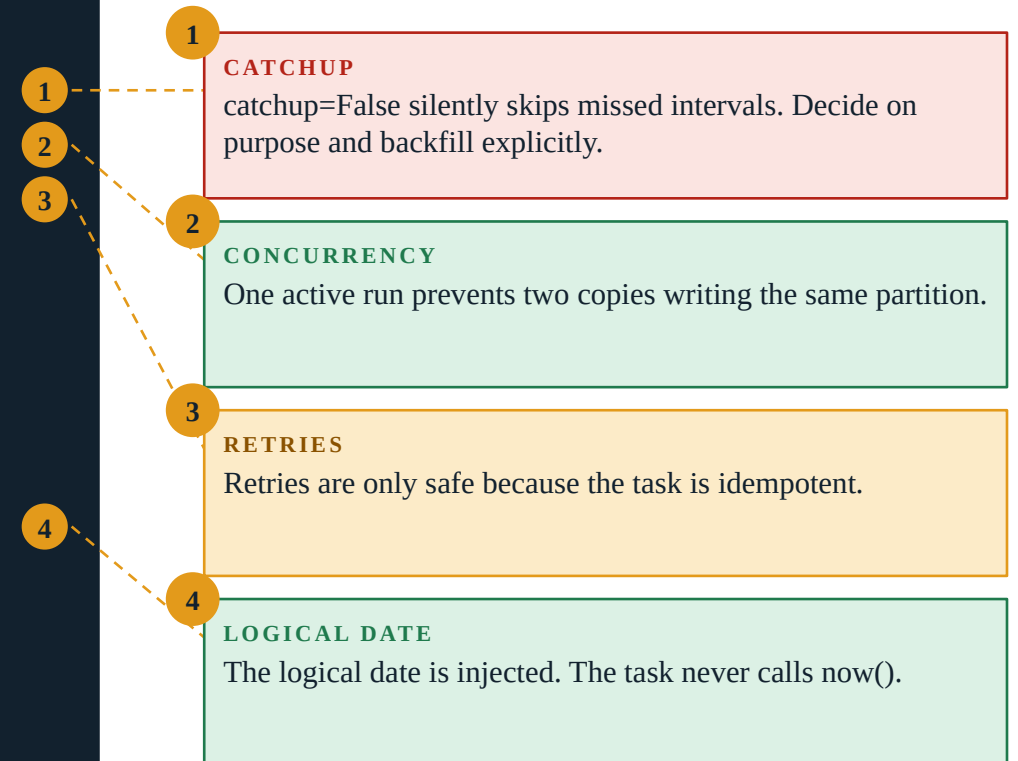
Producer idempotence and broker transactions reduce duplicates inside the log. They do not cover your database sink, so the sink still needs its own guard.

Orchestrator settings that break idempotency

An Airflow-style DAG. Defaults are tuned for convenience, not safety.

```
with DAG(  
    dag_id="orders_daily",  
    schedule="0 2 * * *",  
    start_date=datetime(2026, 1, 1),  
    catchup=False,  
    max_active_runs=1,  
    default_args={"retries": 2,  
                  "retry_delay": timedelta(minutes=5)},  
) as dag:  
    load = PythonOperator(  
        task_id="load_day",  
        python_callable=load_day,  
        op_kwargs={"day": "{{ ds }}"})
```

Python



Reproducible ML training: pin five things

A model is a function of data, code, configuration, randomness and environment.

PIN 1**Data snapshot**

Train on an immutable table version or manifest of files, never on "latest".

PIN 2**Code version**

Record the git SHA. Refuse to train from a dirty working tree.

PIN 3**Configuration**

Hyperparameters and feature lists live in a versioned config, not in a notebook cell.

PIN 4**Randomness**

- Seed every source: split, sampling, initialisation, shuffling.
- Log the seeds.

PIN 5**Environment**

Lock dependencies and use a container image digest.

CAVEAT**Hardware nondeterminism**

- Parallel reductions and GPU kernels can vary in low-order digits.
- Define "same" as within tolerance.

RULE

Score predictions idempotently too: key each row by entity, model version and as-of date.

A run ledger makes every run explainable

One append-only table answers "what produced this?" and "did it finish?"

```
CREATE TABLE ops.run_ledger (  
  run_id STRING,  
  job_name STRING,  
  logical_date DATE,  
  code_sha STRING,  
  input_ref STRING,  
  status STRING,  
  rows_written BIGINT,  
  output_hash STRING,  
  started_at TIMESTAMP,  
  finished_at TIMESTAMP  
);
```

SQL

1

1

RUN ID

run_id is stamped on every output row, so any row traces back to a run.

2

2

PROVENANCE

code_sha and input_ref pin what produced the output.

3

3

OUTPUT HASH

An order-independent hash lets you prove a rerun matched the first run.

NOTE

The ledger is also the source for freshness, duration and volume metrics in Part 6.

Testing idempotency in CI

The run-twice test as a pytest, with a crash variant.

Python

```
def fingerprint(table):  
    df = read(table).sort_values(KEYS)  
    return len(df), hash_rows(df)  
  
def test_run_twice(spark_fixture):  
    run_job(day="2026-09-19", src=FIXTURE)  
    first = fingerprint("gold.daily_orders")  
    run_job(day="2026-09-19", src=FIXTURE)  
    assert fingerprint("gold.daily_orders") == first  
  
def test_crash_then_rerun(spark_fixture):  
    with pytest.raises(SimulatedCrash):  
        run_job(day=D, src=FIXTURE, crash_after="write")  
    run_job(day=D, src=FIXTURE)  
    assert fingerprint("gold.daily_orders") == EXPECTED
```

1

1

FINGERPRINT

Compare counts and an order-independent hash of the sorted rows.

2

2

RUN TWICE

Same fixture, same date, same result.

3

3

CRASH TEST

Inject a failure between write and commit, then rerun. Most real bugs live here.

Gate 1: idempotency review

A pipeline passes when every box can be answered with evidence.

Design

- ☐ Every write is overwrite-by-partition or keyed merge
- ☐ Logical date is an input; no now() in transforms
- ☐ Keys are derived from business keys
- ☐ Late and deleted records have a defined rule

Side effects

- ☐ External calls use idempotency keys or an outbox
- ☐ Offsets commit only after the sink write
- ☐ Concurrent runs of the same slot are prevented
- ☐ Retries are enabled only for safe tasks

Evidence

- ☐ Run-twice and crash tests run in CI
- ☐ Every output row carries run_id
- ☐ Run ledger records code SHA and input ref
- ☐ Training pins data, code, config and seeds

RULE

If a box cannot be ticked, the job is not ready to be retried automatically. Turn retries off until it is.

03

Backfills and reprocessing

Re-running history safely is the hardest routine operation in a data platform.

SLIDES 35-54 · DEPTH 2-3

IN THIS PART

- A taxonomy of backfills and their hazards
- The lifecycle: scope, plan, canary, waves, validate, publish
- Ledgers, isolation and bounded concurrency
- Validation and diff queries
- Point-in-time correctness for ML features
- Estimating cost and time
- Gate 2 checklist

What kind of backfill is this?

The type decides the risk profile, so name it in the plan.

Type	Trigger	Typical scope	Main risk
Catch-up	Missed or failed scheduled runs	A few partitions	Duplicates if the job is not idempotent
Late-data restatement	A source corrects its history	Recent weeks	Changing numbers that people already used
Logic change	A new business rule or bug fix	Months to years	Cascading rebuilds and cost
Schema or new column	The model gains a field	The whole table	Long runs; dual-write complexity
Migration	New platform or file format	Everything	Parity drift between old and new
Model re-score	A new model version	Prediction history	Metric breaks that confuse consumers

PRACTICE

Catch-up backfills are routine. The rest change history, so they need announcements and diffs.

Why backfills are dangerous

Six hazards, each with a matching control later in this part.

HAZARD**1**

Starving production

- Backfill tasks compete with the daily run for compute.
- Control: isolated pool, bounded concurrency.

HAZARD**2**

Runaway cost

- Years of partitions at full scan price, multiplied by retries.
- Control: estimate first; budget cap.

HAZARD**3**

Downstream cascade

- Every dependent table, dashboard and model may need a rebuild.
- Control: lineage-based scoping.

HAZARD**4**

Source drift

- The source no longer returns what it returned then.
- Control: retention and restatement checks.

HAZARD**5**

Partial state

- A crash leaves some partitions new and some old.
- Control: ledger and atomic publish.

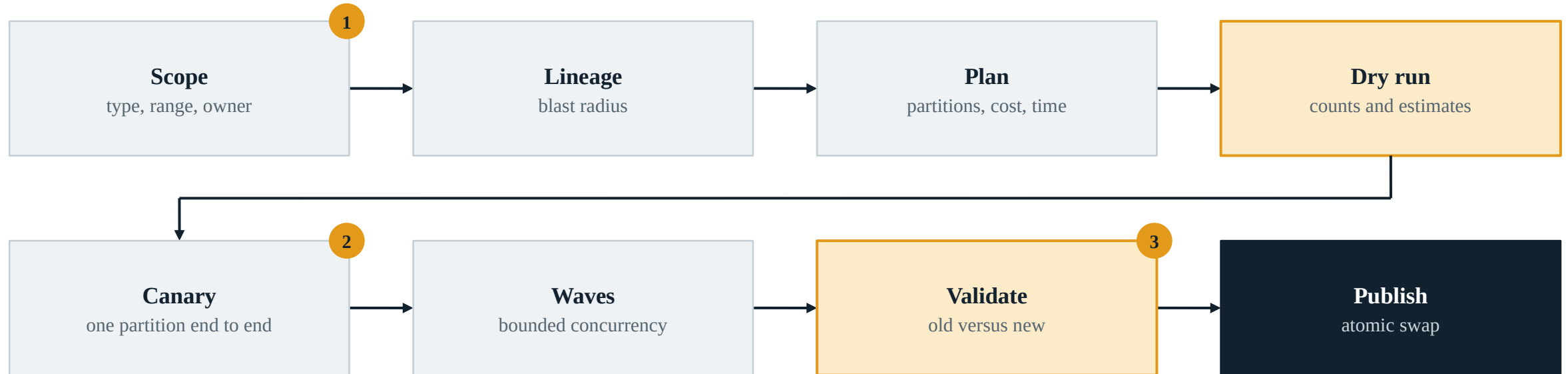
HAZARD**6**

Silent history change

- Numbers people already reported now differ.
- Control: diffs, announcements, retention of old versions.

The backfill lifecycle

Eight stages, three gates. Skipping a stage is how incidents start.



1

NOTE

Scope names the backfill type, the exact range and a single accountable owner.

2

PRACTICE

Run one representative partition end to end before scaling out.

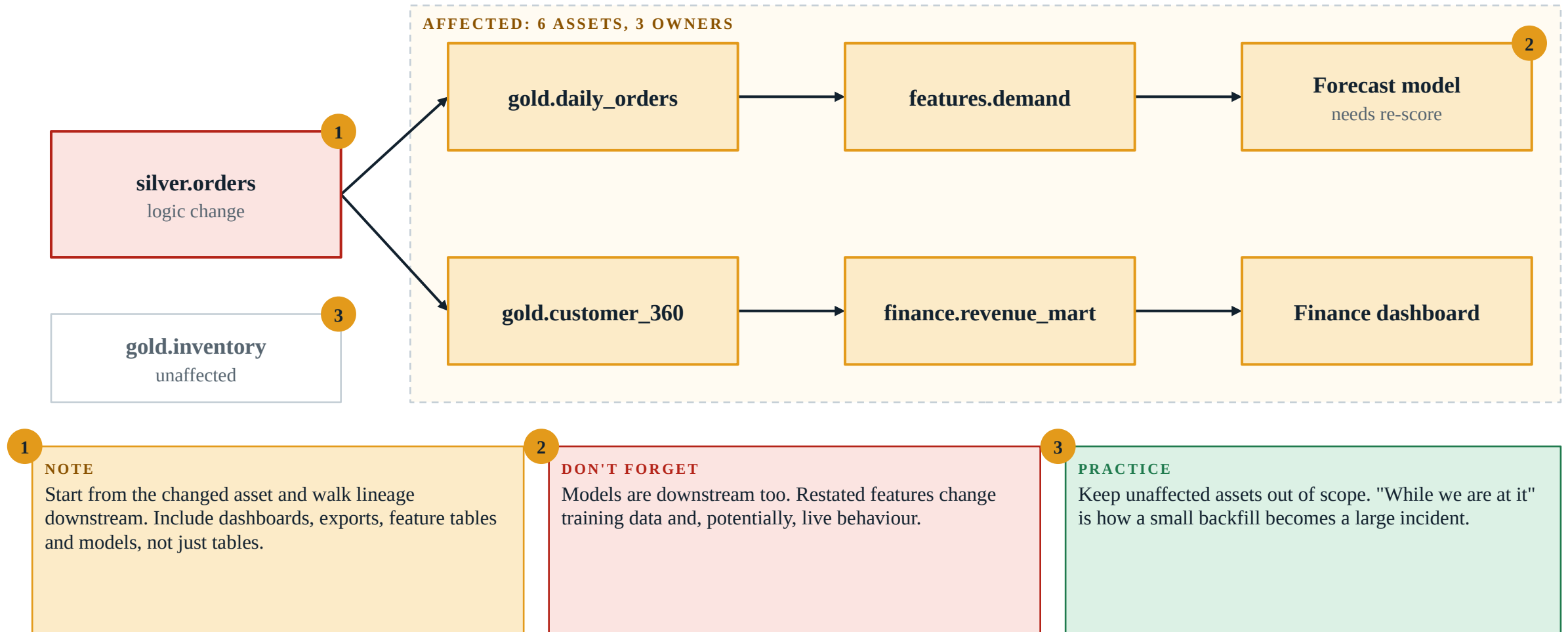
3

GATE

Nothing is visible to consumers until validation passes.

Scoping the blast radius from lineage

Change one silver table and six assets are affected. Find them before you run.



The one-page backfill plan

Write it before you run it. Reviewers approve the page, not the code.

Field	What to write	Example
Type and reason	Backfill type and why now	Logic change: fix tax rounding
Range	First and last partition, inclusive	2022-09-20 to 2026-09-19
Blast radius	Downstream assets and owners	7 tables, 3 dashboards, 1 model
Estimate	Partitions, volume, cost, wall clock	1,461 partitions · 263 TB · about £1,050 · 11 h
Isolation	Where writes land; which compute pool	shadow.orders; pool "backfill", max 20 slots
Validation	Checks and tolerances	Totals within 0.01%; only tax columns change
Rollback	How to undo, and for how long	Keep the old snapshot for 14 days
Approvals	Who signs off	Data owner; Finance for revenue tables

RULE

If the plan does not fit on one page, split the backfill.

Planning: partitions, cost and time

A dry run that costs nothing and prevents expensive surprises.

Python

```
def partitions(start, end):
    d = start
    while d <= end:
        yield d
        d += timedelta(days=1)

plan = list(partitions(date(2022, 9, 20),
                      date(2026, 9, 19)))
scan_tb = len(plan) * 0.18
cost_gbp = scan_tb * 4.0      # illustrative
hours = len(plan) * 9 / 20 / 60

assert cost_gbp <= APPROVED_BUDGET_GBP
```

1

PLAN

Enumerate the exact partitions. The list becomes the ledger.

1

2

COST

Use measured bytes per partition from a sample, and your real price per TB.

2

3

3

TIME

9 minutes each, 20 in parallel. Concurrency is a cost and risk dial.

4

4

BUDGET GUARD

Fail fast when the estimate exceeds what was approved.

A ledger makes backfills resumable

One row per partition. Workers claim work atomically.

```
CREATE TABLE ops.backfill_ledger (  
  backfill_id STRING, partition_key STRING,  
  status STRING, attempt INT, code_sha STRING,  
  rows_in BIGINT, rows_out BIGINT, claimed_by STRING,  
  started_at TIMESTAMP, finished_at TIMESTAMP);
```

-- a worker claims the next pending partition

```
UPDATE ops.backfill_ledger  
SET status = 'running', claimed_by = :worker,  
    attempt = attempt + 1  
WHERE backfill_id = :id  
    AND status = 'pending'  
    AND partition_key = :candidate;
```

SQL

1

1

STATE

Status per partition: pending, running, done, failed. Restarts skip done rows.

2

CLAIM

The status check inside the UPDATE is the lock. Check rows affected.

2

3

3

RACE

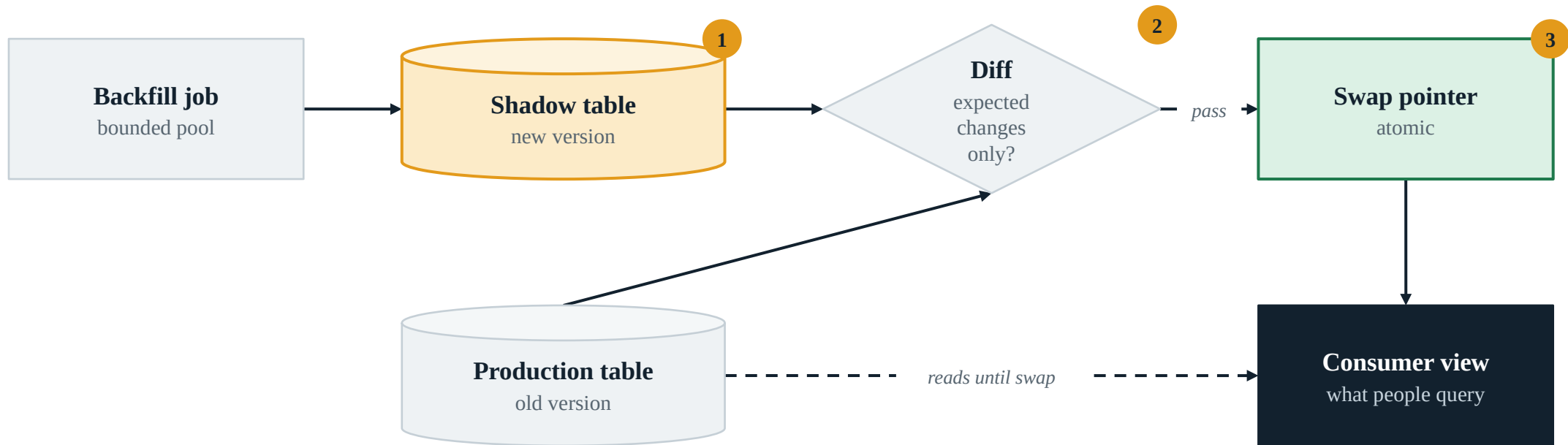
If zero rows changed, another worker won. Pick a different candidate.

NOTE

Because each partition write is idempotent, a crashed worker can be retried safely: reset its row to pending.

Isolation: write a shadow, then swap

Production keeps serving the old data until the new data has earned trust.



1

NOTE

Table formats make shadows cheap: Iceberg branches, Delta clones or a plain table with a version suffix.

2

PRACTICE

Run the diff query (slide 48) on the shadow against production. Only declared changes may appear.

3

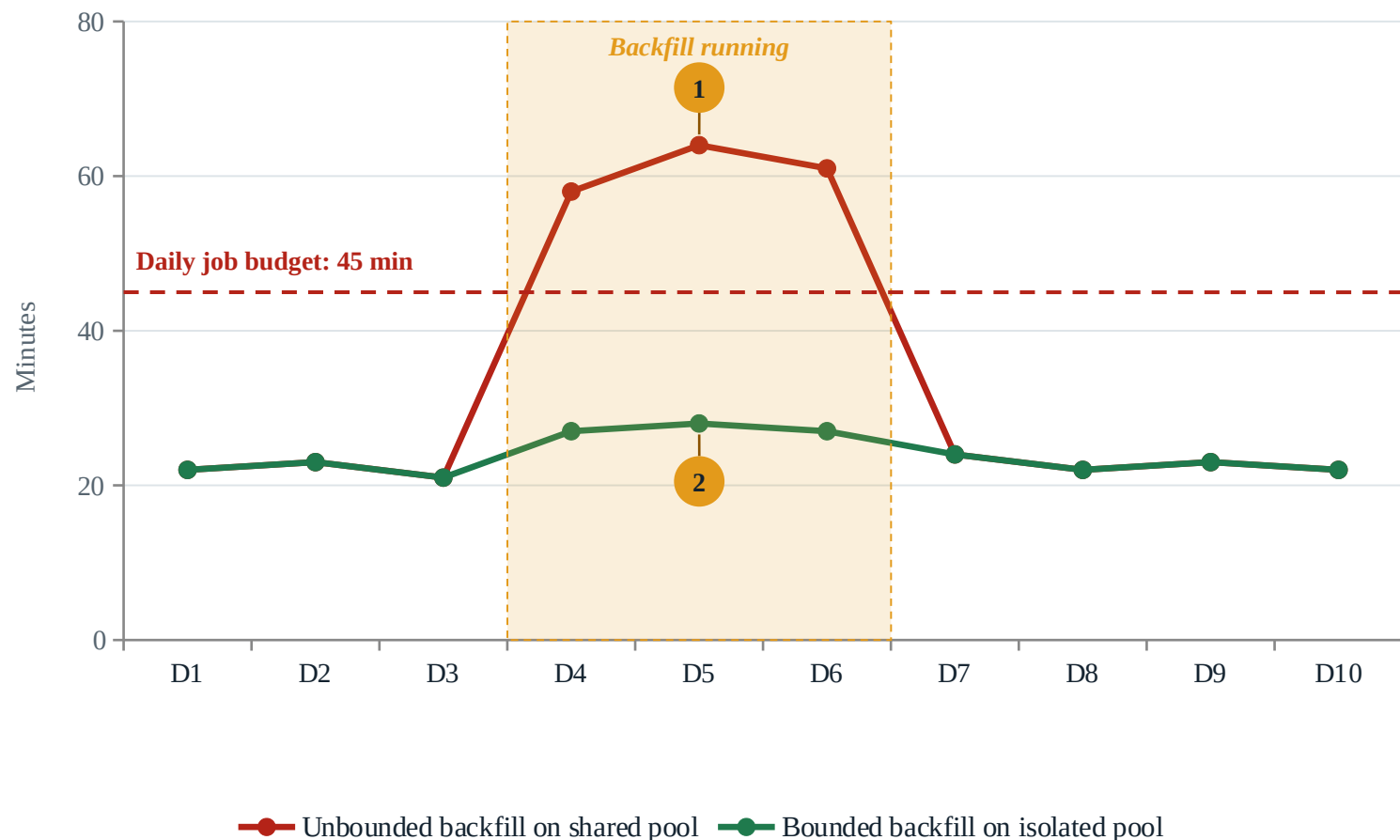
PRACTICE

The swap is a pointer change, so rollback is the same operation in reverse.

Bounded concurrency protects the daily run

Same backfill, two configurations. Only one leaves the morning pipeline alone.

Figure 2. Daily job duration in minutes across ten days (illustrative)



1

PITFALL

Unbounded: the daily job takes 64 minutes and misses its freshness deadline.

2

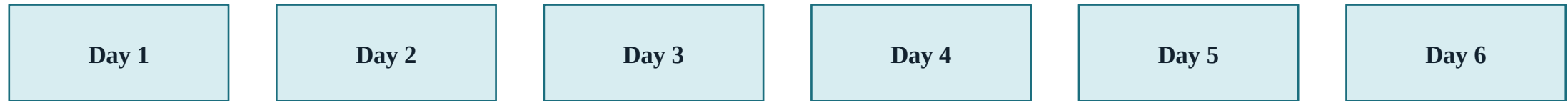
PRACTICE

Bounded and isolated: 28 minutes. Slower backfill, safe morning.

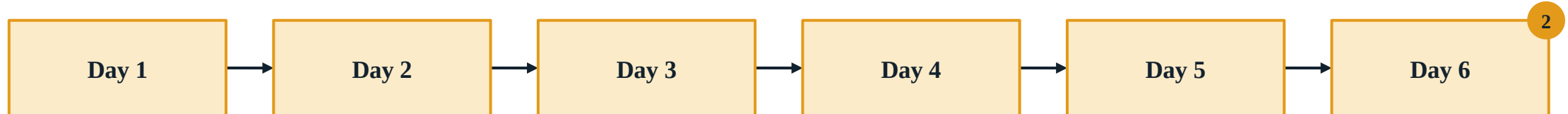
Ordering and dependencies between partitions

Parallelism is only free when partitions do not depend on each other.

INDEPENDENT PARTITIONS: RUN IN PARALLEL UP TO THE CAP



STATEFUL PARTITIONS: EACH NEEDS THE PREVIOUS OUTPUT



1

PRACTICE

Independent partitions can run in parallel, up to the concurrency cap you set for safety and cost.

2

SEQUENTIAL

Running totals, sessionisation and slowly changing dimensions must run in order. Extra parallelism buys nothing and can corrupt results.

DESIGN FIX

Prefer logic that recomputes from a bounded window of raw data rather than from yesterday's output. It parallelises and it heals.

Sources push back: constraints to check first

A backfill is a load test on every system it reads from.

Source type	Risk during a backfill	Mitigation
Operational DB replica	Heavy scans slow the application	Read a snapshot or replica; cap concurrency; run off-peak
Third-party API	Rate limits, per-call cost, quota lockout	Throttle; cache raw responses; resume from the ledger
Event log or topic	Retention expired, so the data is gone	Check retention before planning; replay from archive
"Current state" endpoint	Returns today's value, not history	Rebuild from snapshots you kept; document the gap
Vendor files	Restated files replace old ones	Version by arrival time; keep originals immutable
Upstream tables	Another team changes them mid-run	Read a pinned snapshot; agree a freeze window

PITFALL

If history cannot be recreated faithfully, say so. A partial backfill presented as complete is worse than a documented gap.

What to compare: old versus new

Six checks catch most backfill defects before consumers see them.

Check	Query idea	Tolerance	Catches
Row counts	count(*) per partition, old versus new	0% unless logic changes rows	Lost or duplicated rows
Key set	Anti-join on the key in both directions	No unexpected keys	Dropped or phantom entities
Totals	sum(amount) per day	Within 0.01%	Fan-out joins; dropped rows
Null rates	Share of nulls per column	Within 0.5 points	Broken joins; bad defaults
Distributions	Quantiles or PSI per column	PSI below 0.1	Unit or time-zone shifts
Declared changes	Which columns differ, versus the plan	Only declared columns	Accidental side effects

PRACTICE

Write tolerances into the plan before the run. Deciding after you see the numbers invites rationalisation.

The diff query

Classify every key as added, removed, changed or same.

```
WITH d AS (  
  SELECT coalesce(n.order_id, o.order_id) AS order_id,  
  CASE  
    WHEN o.order_id IS NULL THEN 'added'  
    WHEN n.order_id IS NULL THEN 'removed'  
    WHEN n.net_revenue IS DISTINCT FROM  
      o.net_revenue THEN 'changed'  
    ELSE 'same' END AS change  
  FROM shadow.orders n  
  FULL OUTER JOIN gold.orders o  
    ON n.order_id = o.order_id  
  WHERE :day IN (n.order_date, o.order_date))  
SELECT change, count(*) AS n FROM d GROUP BY change;
```

SQL

1

FULL OUTER

A full outer join sees keys present on only one side.

1

2

NULL-SAFE

IS DISTINCT FROM treats NULL correctly. Plain <> would hide null changes.

2

3

SCOPE

Restrict to one partition so the diff itself stays cheap.

3

RULE

Extend the CASE with one branch per column in scope. Any change outside the declared set fails the gate.

Publishing: the release procedure

A backfill is a release. Treat the swap like a production deploy.

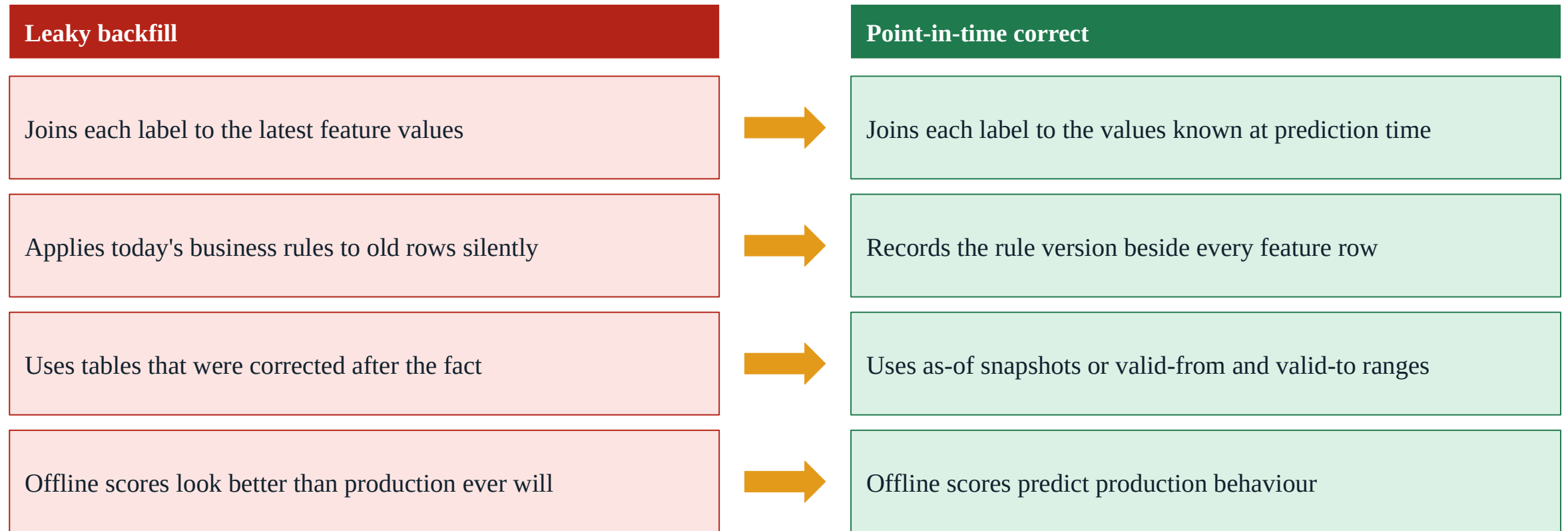
1	Announce	Tell affected owners what changes, when, and how to recognise it.	CHECK <i>Notice sent a day ahead; owners acknowledged</i>
2	Freeze	Pause competing writers and downstream refreshes on the target.	CHECK <i>Schedules paused; no active runs on the target</i>
3	Swap	Point production at the audited shadow in one metadata operation.	CHECK <i>Old and new snapshot ids recorded</i>
4	Verify	Re-run counts and reconciliation on production, not on the shadow.	CHECK <i>Counts, totals and key set match the plan</i>
5	Release	Resume schedules, trigger downstream rebuilds, refresh dashboards.	CHECK <i>Downstream runs green; consumers notified</i>
6	Retain	Keep the old version for a rollback window, then clean up.	CHECK <i>Old snapshot kept 14 days; ledger closed</i>

TIMING

Avoid swap windows near reporting deadlines. Rollback must be one command, tested in advance.

Backfilling features without leaking the future

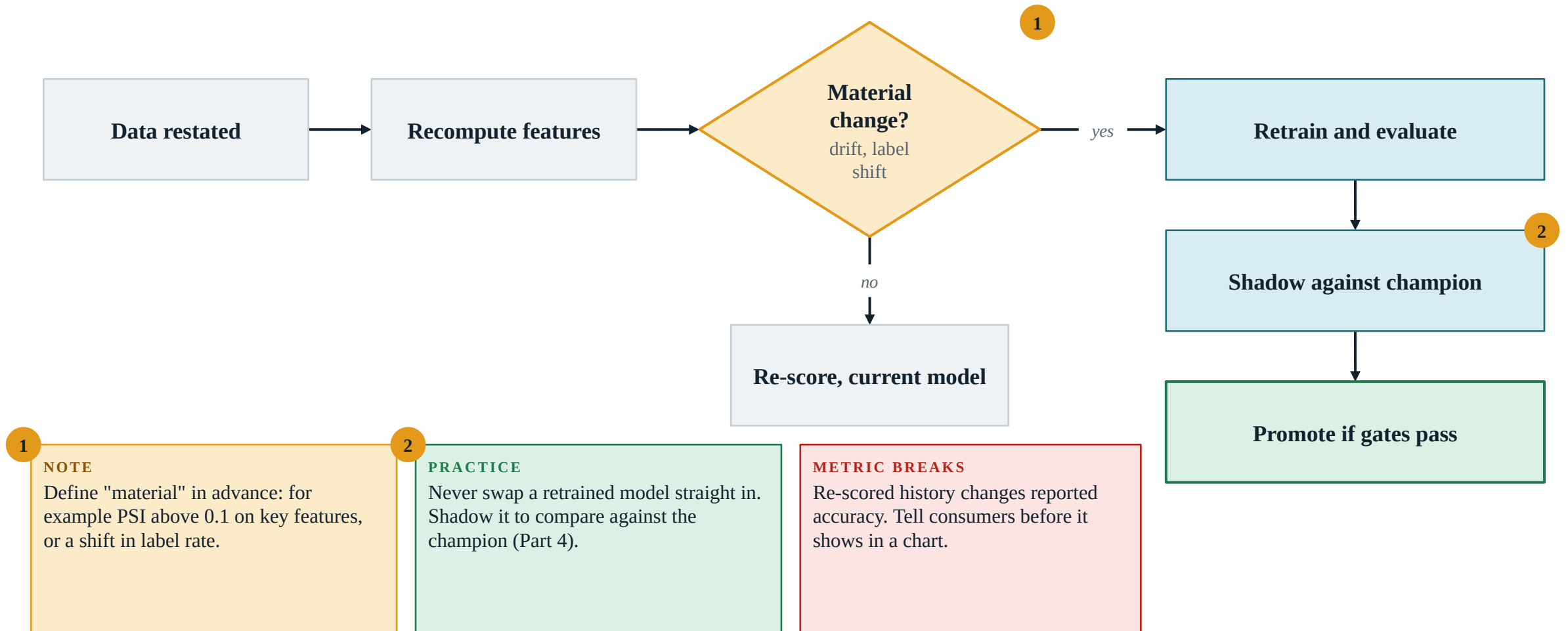
Point-in-time correctness is the ML-specific trap.

**RULE**

A feature row may only use data whose event time is at or before the prediction time.

After a backfill: do models need retraining?

Restated history can change features, labels and live behaviour.



Estimate before you run

Illustrative estimate for the running example: four years of daily partitions.

1,461

partitions

four years of daily data

263 TB

to scan

1,461 partitions × 0.18 TB

≈ £1,050

scan cost

at an illustrative £4 per TB

≈ 11 h

wall clock

9 minutes each, 20 in parallel

Concurrency	Wall clock	Pressure on shared systems
10 slots	about 22 hours	Gentle; slow enough to run for a day
20 slots	about 11 hours	Moderate; fits an overnight window
40 slots	about 5.5 hours	High; may throttle sources or starve neighbours

Backfill anti-patterns and what to do instead

Depth 3 begins. Each anti-pattern below has caused a real incident somewhere.

Anti-pattern		Practice
Re-run the DAG from 2019 in production and hope	➡	Plan partitions, cost and blast radius first
Backfill on the production cluster at 09:00	➡	Isolated pool, bounded concurrency, off-peak window
Overwrite gold in place	➡	Write a shadow, diff it, swap atomically
Track progress in a spreadsheet or in your head	➡	A ledger with per-partition status; resumable
Tell consumers afterwards	➡	Announce first; notify again on release
One giant run; any failure means restart	➡	Small idempotent partitions; retry only failures

Gate 2: backfill readiness

Do not press run until every box has evidence.

Plan

- ☐ Type, range and owner recorded
- ☐ Lineage and blast radius documented
- ☐ Partitions, cost and time estimated
- ☐ Source retention and limits checked

Execute

- ☐ Writes go to a shadow or branch
- ☐ Concurrency bounded on an isolated pool
- ☐ Ledger tracks every partition
- ☐ Canary partition verified end to end

Publish and learn

- ☐ Old versus new diff reviewed
- ☐ Swap is atomic and reversible
- ☐ Downstream rebuilds and models handled
- ☐ Owners notified; retrospective written

RULE

A backfill is a release with a bigger blast radius. It earns the same rigour as a production deploy.

04

CI/CD for data and ML

Ship pipeline, contract and model changes behind automated gates.

SLIDES 55–76 • DEPTH 3

IN THIS PART

- Why data CI differs from software CI
- The test pyramid, contracts and schema evolution
- Ephemeral environments, slim CI and data diffs
- A complete workflow with short-lived credentials
- Deployment strategies and expand/contract
- ML pipelines: gates, shadowing and rollback
- Delivery metrics and Gate 3

Why data CI/CD is different

Software CI asks "does it run?". Data CI must also ask "is the output right?".

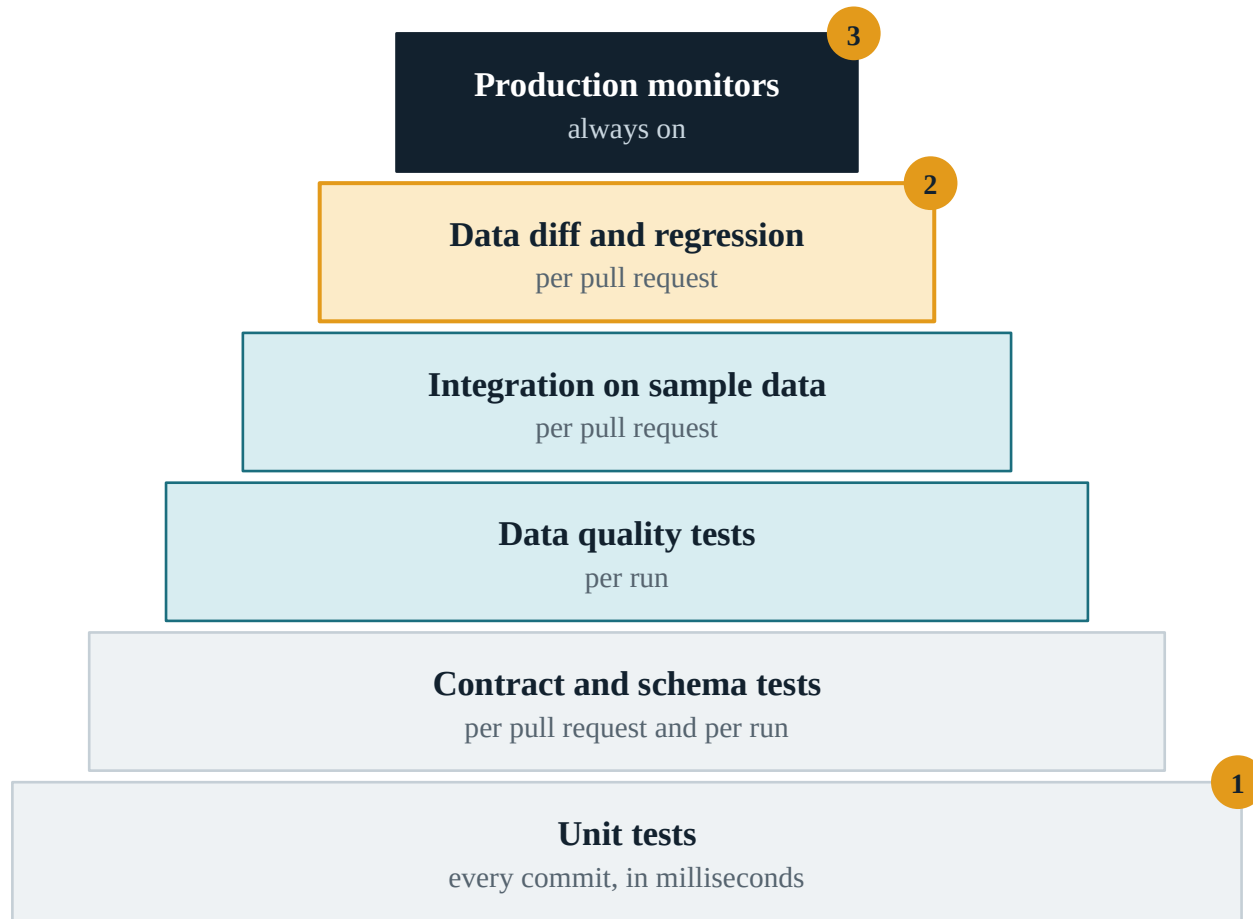
Dimension	Software	Data and ML
What changes	Code	Code, data, schemas, models and infrastructure
What tests need	Small deterministic fixtures	Representative data, sometimes large or sensitive
"Correct" means	Assertions pass	Assertions pass and output looks right against history
Environments	Cheap to clone	Data is large; clones need cost and privacy care
Typical failure	A crash	Silent: wrong numbers with green checks
Rollback	Redeploy the previous build	Code, data and model may all need rolling back

RULE

A green build shows the code is plausible. Only data checks and diffs show that the output is.

The data test pyramid

Many cheap tests at the base; fewer, costlier checks as you climb; monitors above them all.



3

NOTE

Monitors (Part 6) catch what CI cannot: real-world data that differs from anything in a fixture.

2

PRACTICE

Data diffs compare the PR output to production. They turn "does this look right?" into a reviewable artifact.

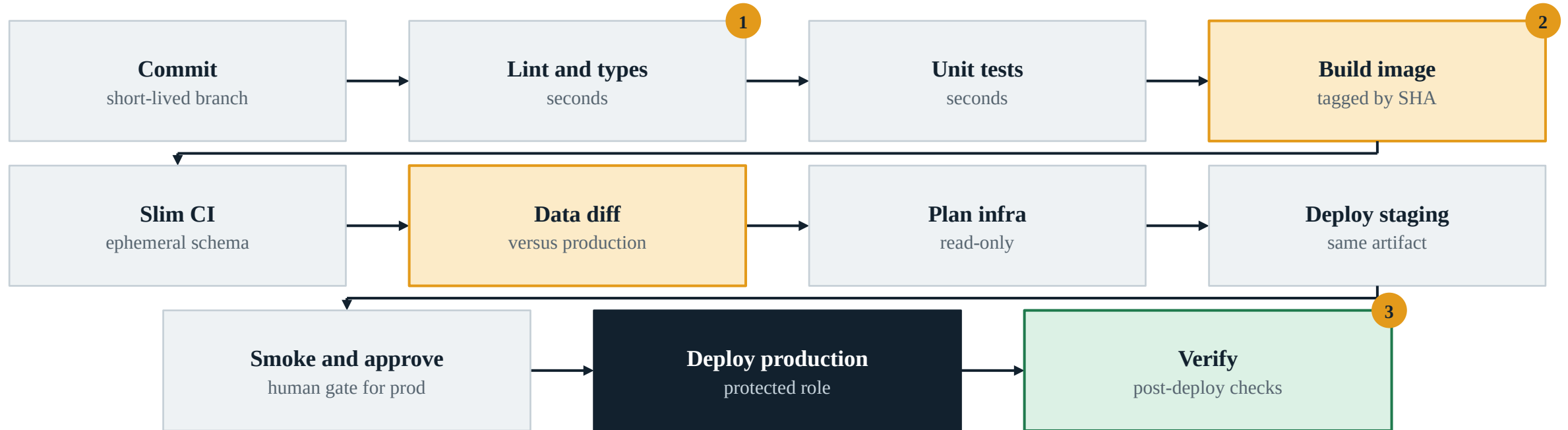
1

NOTE

Invest most in unit tests of transformation logic. They are fast, deterministic and pinpoint the fault.

Pipeline stages, cheapest check first

Eleven stages from commit to verified production.



1

NOTE

Fast feedback first. Cheap checks fail before expensive ones burn compute.

2

PRACTICE

Build once, promote the same artifact through every environment.

3

DEFINITION OF DONE

Post-deploy verification is part of the pipeline, not a hope.

Static checks: lint everything, locally and in CI

A pre-commit configuration covering Python, SQL, Terraform and secrets.

YAML

```
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: PIN_A_RELEASE_TAG
  hooks: [{id: ruff}, {id: ruff-format}]
- repo: https://github.com/sqlfluff/sqlfluff
  rev: PIN_A_RELEASE_TAG
  hooks: [{id: sqlfluff-lint}]
- repo: https://github.com/antonbabenko/pre-commit-terraform
  rev: PIN_A_RELEASE_TAG
  hooks: [{id: terraform_fmt}, {id: terraform_validate}]
- repo: https://github.com/Yelp/detect-secrets
  rev: PIN_A_RELEASE_TAG
  hooks: [{id: detect-secrets}]
```

1

PIN

Pin every hook to a released tag. Floating versions make lint results drift.

1

2

2

PARITY

The same linter versions run in CI, so a local pass predicts a CI pass.

3

SECRETS

Catch secrets before they enter git history. Rewriting history is painful.

3

NOTE

PIN_A_RELEASE_TAG is a placeholder. Choose current release tags for your own repository.

Unit tests for transformation logic

State the business rule in the test, using a fixture small enough to read.

Python

```
def test_keeps_latest_version(spark):  
    src = spark.createDataFrame(  
        [("o1", "10:00", 10.0),  
         ("o1", "10:05", 12.0),  
         ("o2", "09:00", 5.0)],  
        ["order_id", "updated_at", "amount"])  
    out = latest_per_order(src).collect()  
    got = {r.order_id: r.amount for r in out}  
    assert got == {"o1": 12.0, "o2": 5.0}
```

```
def test_empty_input_gives_empty_output(spark):  
    out = latest_per_order(empty_orders(spark))  
    assert out.count() == 0
```

1

FIXTURE

Hand-built rows make the rule visible: two versions of o1, latest wins.

1

2

ASSERT

Assert on business meaning, not on row order or column position.

2

3

EDGES

Test the edges: empty input, nulls, duplicates, late versions, DST days.

3

PRACTICE

Write transformations as pure functions (DataFrame in, DataFrame out). They then test without any infrastructure.

Data quality tests, declared next to the model

A dbt-style schema file. Severity separates blockers from warnings.

YAML

```
models:
- name: gold_daily_orders
  columns:
    - name: order_id
      tests: [unique, not_null]
    - name: status
      tests:
        - accepted_values:
            values: [placed, shipped, cancelled]
- name: customer_id
  tests:
    - relationships:
        to: ref('dim_customer')
        field: customer_id
        config: {severity: warn}
```

1

KEYS

Uniqueness and not-null on the key protect every downstream join.

2

DOMAIN

Accepted values catch new upstream statuses before consumers do.

3

SEVERITY

Use warn for known-imperfect data you are tracking. Use error for blockers.

NOTE

Recent dbt releases prefer the key `data_tests` over `tests`. Check the syntax for your version.

Data contracts: the interface between teams

A machine-readable promise that CI can enforce on both sides.

YAML

```
contract: gold.daily_orders
version: 2.1.0
owner: team-commerce-data
schema:
  - name: order_id
    type: string
    required: true
    unique: true
  - name: net_revenue_gbp
    type: decimal(18,2)
    description: Revenue after discounts, before VAT
sla:
  freshness: before 06:00 Europe/London
  completeness: at least 99.9% of source orders
  breaking_changes: major version and 30 days notice
```

1

VERSION

Semantic versioning tells consumers how careful to be.

1

2

SEMANTICS

Describe meaning and units. Most silent breaks are semantic, not structural.

2

3

SLA

The SLA in the contract is the SLO from Part 1, written where consumers see it.

3

NOTE

The format is illustrative. Use an open contract standard or your platform's native contract feature.

Schema evolution: safe and unsafe changes

Classify every change before merging it.

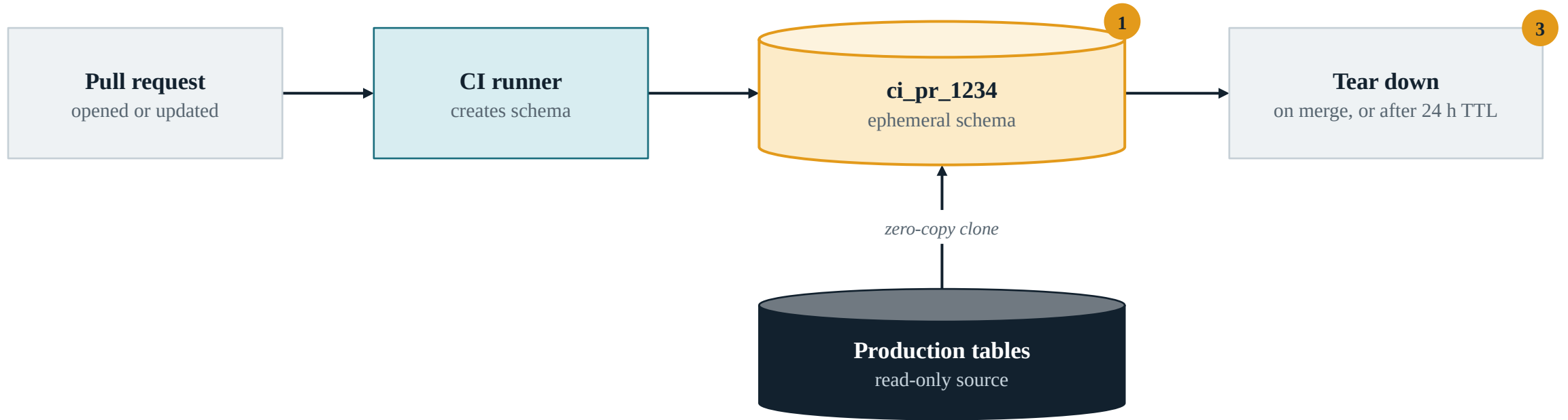
Change	Safe for existing readers?	Version	What to do
Add a nullable column	Yes	Minor	Note it in the changelog
Add a required column	Yes for readers; producers must supply it	Minor	Backfill before enforcing not-null
Widen a type (int to bigint)	Usually	Minor	Test readers with strict casts
Rename a column	No	Major	Expand and contract: add, dual-write, migrate, remove
Remove a column	No	Major	Deprecate for the notice period first
Change meaning or unit, same name	No, and silently	Major	Never reuse a name; add a new column
Narrow a type or tighten a constraint	No	Major	Validate all existing data first

PITFALL

The dangerous change is the one that keeps the same name and shape but changes the meaning. No parser catches it.

Ephemeral environments: one schema per pull request

Isolated, realistic, and deleted automatically.



1

PRACTICE

One schema per PR keeps runs isolated, and posts tests and the data diff back to the PR. No "who broke staging".

2

DATA GOVERNANCE

Clones expose production data to CI. Mask or sample sensitive columns, and restrict who can read the schema.

3

NOTE

A TTL job deletes leftovers. Orphaned schemas are a quiet cost leak.

Slim CI: build only what changed

Compare against production state so CI stays fast and cheap.

Shell

```
# fetch the production manifest to compare against
aws s3 cp s3://meridian-artifacts/prod/manifest.json \
  ./prod-state/
```

```
# build changed models and everything downstream
dbt build \
  --select "state:modified+" \
  --defer --state ./prod-state \
  --target ci
```

```
# guard scan volume (custom macro)
dbt run-operation assert_dry_run_under_budget \
  --args '{limit_gb: 500}'
```

1

SELECT

state:modified+ selects changed nodes plus their descendants.

1

2

2

DEFER

--defer reads unchanged upstream models from production instead of rebuilding them.

3

3

COST GUARD

A cost guard in CI stops an expensive change before it merges. The macro is your own.

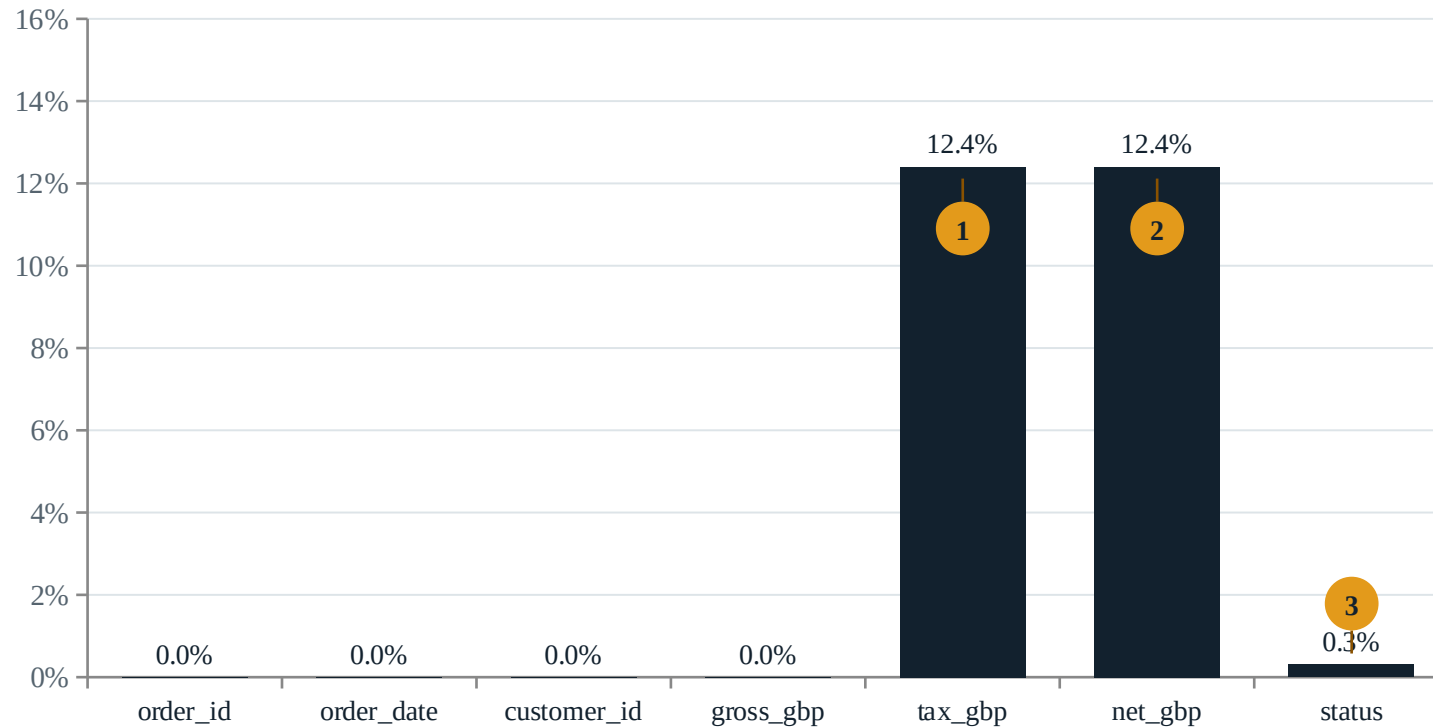
NOTE

Commands follow dbt conventions. Other tools offer equivalents; the principle is to build only the changed subgraph.

Data diff: the change you declared and the change you did not

A pull request should move only the columns it says it moves.

Figure 3. Share of rows that differ from production, by column (illustrative)



1

PRACTICE

Declared: the PR fixes tax rounding. Expected.

2

PRACTICE

Net revenue derives from tax, so it moves too. Expected.

3

PITFALL

Not declared. Block the merge until someone explains why status changed.

A CI workflow with short-lived credentials

GitHub Actions syntax, teaching-sized.

GitHub Actions

```
name: ci
on: pull_request
concurrency:
  group: ci-${{ github.head_ref }}
  cancel-in-progress: true
permissions:
  id-token: write
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ vars.CI_PLAN_ROLE }}
          aws-region: eu-west-2
      - run: make lint unit slim-ci
```

1

CONCURRENCY

A new push cancels the stale run for the same PR: less cost, faster feedback.

2

PERMISSIONS

id-token: write lets the job request an OIDC token. Nothing else is granted.

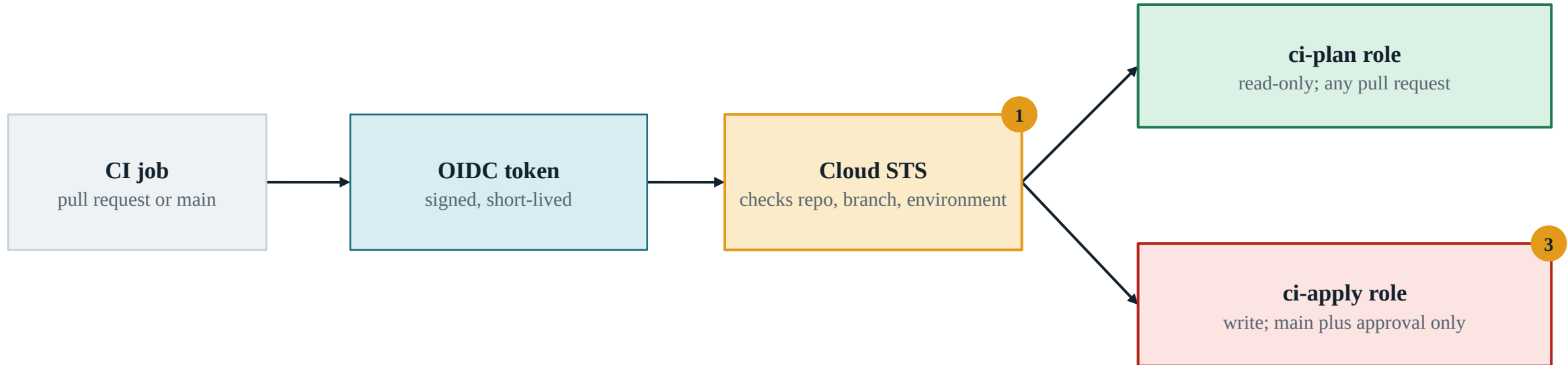
3

IDENTITY

The job assumes a read-mostly role. There are no stored cloud keys.

Identity in CI: no long-lived keys

The job proves who it is; the cloud hands back temporary, narrowly scoped access.



1

PIN THE CLAIMS

The trust policy must name the repository and branch or environment. Unpinned, any repository could assume the role.

PRACTICE

No cloud keys in CI secrets: nothing to leak and nothing to rotate. Tokens expire in minutes.

3

PRACTICE

Split plan and apply roles. A compromised pull request build can read, but not write.

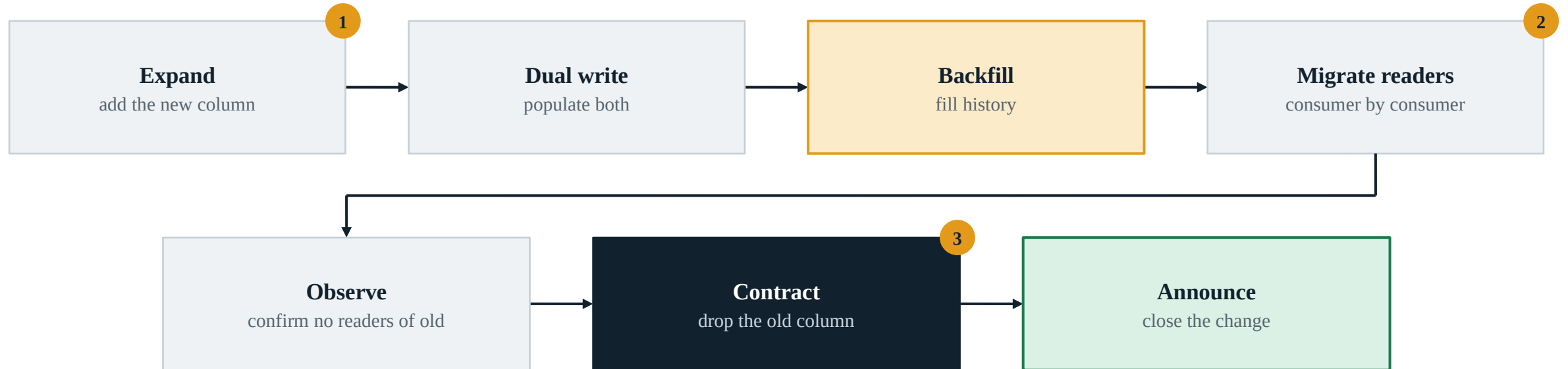
Deployment strategies for data

Pick the strategy by blast radius and reversibility.

Strategy	How it works	Rollback	Best for
Big bang	Replace and rerun everything at once	Redeploy and rebuild	Tiny, low-risk changes
Blue/green tables	Build the new table; swap a view or pointer	Swap back	Gold tables with strict consumers
Canary partitions	Apply new logic to one slice first	Rerun slice, old logic	Uncertain-impact logic changes
Shadow run	Run in parallel; compare but do not publish	Nothing to undo	Migrations and model changes
Expand and contract	Add new beside old; migrate readers; remove old	Stop before contracting	Schema and contract changes
Feature flags	Toggle new behaviour by configuration	Flip the flag	Behaviour in shared code

Expand and contract: renaming a column safely

Breaking changes become a sequence of non-breaking ones.



1

NOTE

Additive changes are safe. Nothing existing breaks.

2

PRACTICE

Track which consumers have moved. Use access logs, not guesses.

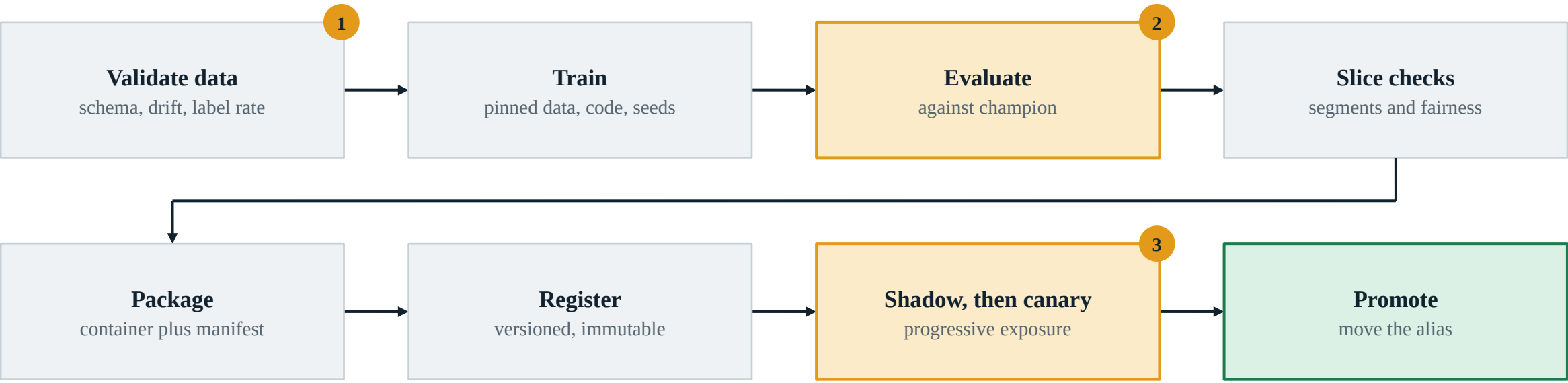
3

GATE

Contract only when logs show no reads of the old column for a full cycle.

An ML delivery pipeline

Training is a build. Promotion is a deploy. Each has gates.



1

PRACTICE

Fail before spending on training if the inputs are broken.

2

NOTE

Compare to the current production model, not to a fixed number from last year.

3

PROGRESSIVE

Exposure grows only while live metrics hold.

Model promotion gates

Illustrative thresholds. Set yours from business tolerance, and write them down.

Gate	Measure	Illustrative threshold	On failure
Data validation	Schema, null rates, label rate	No critical failures	Stop; page data owner
Offline quality	WAPE versus champion	Not worse by more than 1% relative	Reject candidate
Slice quality	WAPE per store cluster	No slice worse by more than 5%	Block; investigate slice
Fairness and policy	Error parity across regions	Within the agreed band	Escalate to model owner
Latency and size	p95 scoring time; artifact size	Within budget	Optimise or reject
Reproducibility	Retrain from manifest matches metric	Within tolerance	Fix the pipeline first

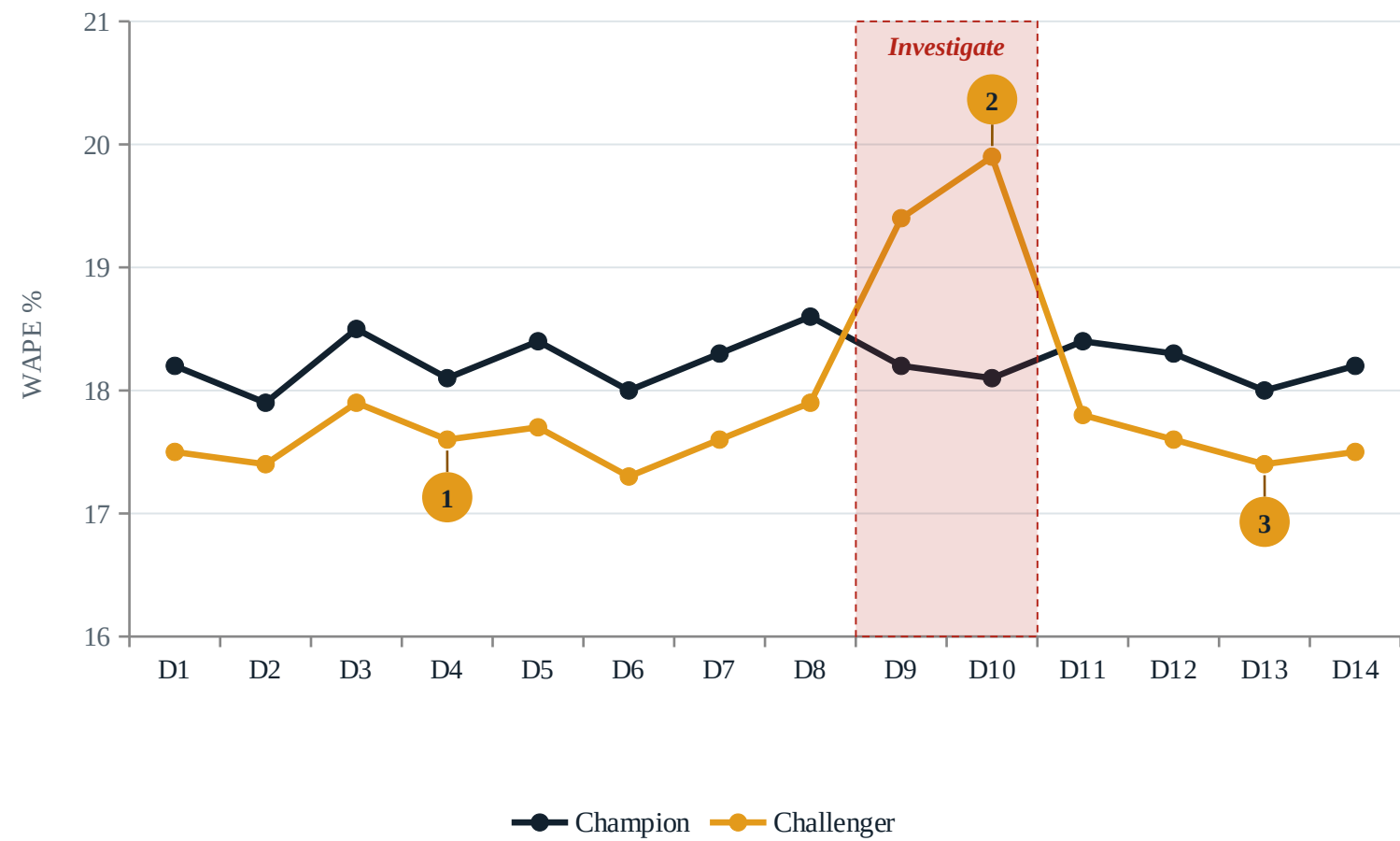
DEFINITION

WAPE (weighted absolute percentage error) = sum of |actual minus forecast| divided by sum of |actual|.

Shadow evaluation: challenger versus champion

Fourteen days of live shadowing. Good on average is not good enough to promote.

Figure 4. Daily WAPE, % (lower is better; illustrative)



1 PRACTICE
The challenger beats the champion on 12 of 14 days, by about 0.6 points.

2 PITFALL
Days 9 and 10 are worse, by up to 1.8 points. Find out why before promoting.

3 DECISION
Illustrative rule: promote only if no single day is worse by more than 1.5 points. Day 10 breaks it, so hold the promotion.

Rollback: one mechanism per artifact

Decide the rollback before you deploy, not during the incident.

Artifact	Mechanism	Speed	Watch out
Pipeline code	Redeploy the previous git tag	Minutes	Config or schema already migrated
Table data	Table-format time travel: restore a version or snapshot	Minutes	Retention window; consumers may have read bad data
Views and pointers	Re-point to the prior version	Seconds	Caches and materialised dependants
Model	Move the registry alias back	Minutes	Feature schema must match the older model
Infrastructure	Revert and apply; or restore state backup	Minutes to hours	Stateful resources may not un-destroy
Stream offsets	Reset consumer group to a timestamp	Minutes	Duplicates; needs idempotent sinks

PRACTICE

Rehearse each rollback in staging. An untested rollback is a hypothesis.

Measuring delivery: DORA metrics for data

Illustrative targets for the running example. Track trends, not absolutes.

daily+
Deployment frequency
small, frequent changes

< 1 day
Lead time for changes
commit to production

< 15%
Change failure rate
deploys causing an incident or rollback

< 1 hour
Time to restore
from detection to recovery

Metric	How to count it for data
Deployment	A merge that changes pipeline, contract, model or infrastructure in production
Failure	A rollback, hotfix, or an incident traced to the change
Restore	Data correct and fresh again, not merely the job green

Gate 3: CI/CD readiness

What a pipeline needs before its changes are allowed to ship automatically.

Pipeline

- ☐ Lint, types and unit tests block merges
- ☐ Build once; promote the same artifact
- ☐ CI uses short-lived, pinned credentials
- ☐ Plan and apply roles are separate

Data and models

- ☐ Contracts versioned and enforced in CI
- ☐ Quality tests with severities
- ☐ Data diff reviewed on every PR
- ☐ Model gates written down and automated

Delivery

- ☐ Ephemeral environments with TTL
- ☐ Rollback documented per artifact
- ☐ Post-deploy verification automated
- ☐ Delivery metrics tracked monthly

RULE

Automation without gates is just faster failure. Gates without automation are just meetings.

05

Terraform for the data platform

Infrastructure changes become reviewed, tested and reversible, like any other change.

SLIDES 77-96 · DEPTH 3-4

IN THIS PART

- What Terraform owns, and what it should not
- Repository, state and blast-radius design
- Modules, environments and promotion
- Plans, drift and safe refactoring
- Policy as code, testing and secrets
- Alarms and cost guardrails as code
- Gate 4 checklist

Where Terraform's responsibility ends

Give every object exactly one owner.

TERRAFORM OWNS

Containers and permissions

- Buckets, networks, clusters, warehouses
- IAM roles and policies
- Queues, topics, secret containers
- Alarms, dashboards, budgets

PIPELINES OWN

Contents and logic

- Table schemas and migrations
- Transformation code and tests
- Model artifacts and registries
- Orchestrator DAG definitions

THE SEAM

Where they meet

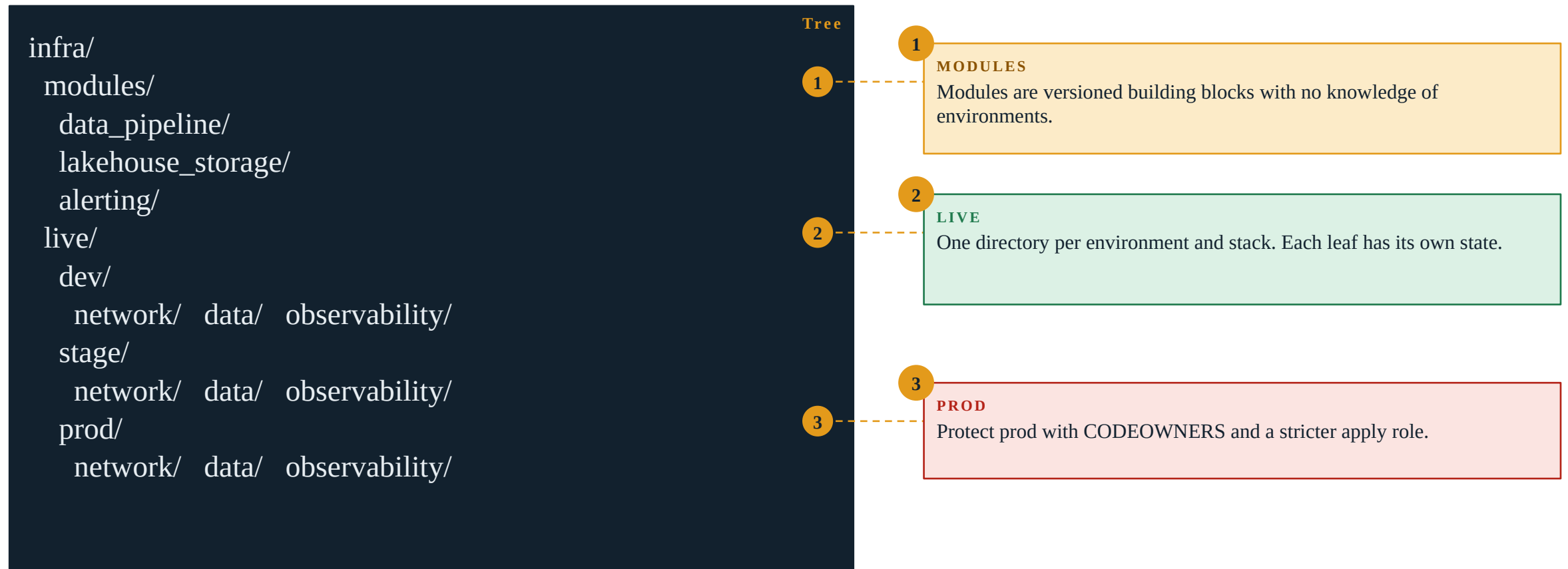
- Roles that pipelines assume
- Names and ARNs handed over as outputs
- Grants on schemas: choose one owner
- Never both managing the same object

RULE

If Terraform and a pipeline both manage an object, they will fight, and drift alerts will never go quiet.

Repository layout

Reusable modules on one side, environment-specific stacks on the other.

**NOTE**

Directories per environment are easier to review than workspaces, and they make blast radius visible in the file tree.

Remote state, encrypted and locked

State is the source of truth about what Terraform manages. Protect it.

```
terraform {  
  required_version = ">= 1.11"  
  
  backend "s3" {  
    bucket = "meridian-tfstate-prod"  
    key = "live/prod/data/terraform.tfstate"  
    region = "eu-west-2"  
    encrypt = true  
    use_lockfile = true  
  }  
}
```

HCL**1****VERSION**

Pin a minimum version so nobody applies with a client that lacks needed features.

2**ENCRYPT**

Encrypt state at rest. It can contain secrets in plaintext.

2**3****3****LOCK**

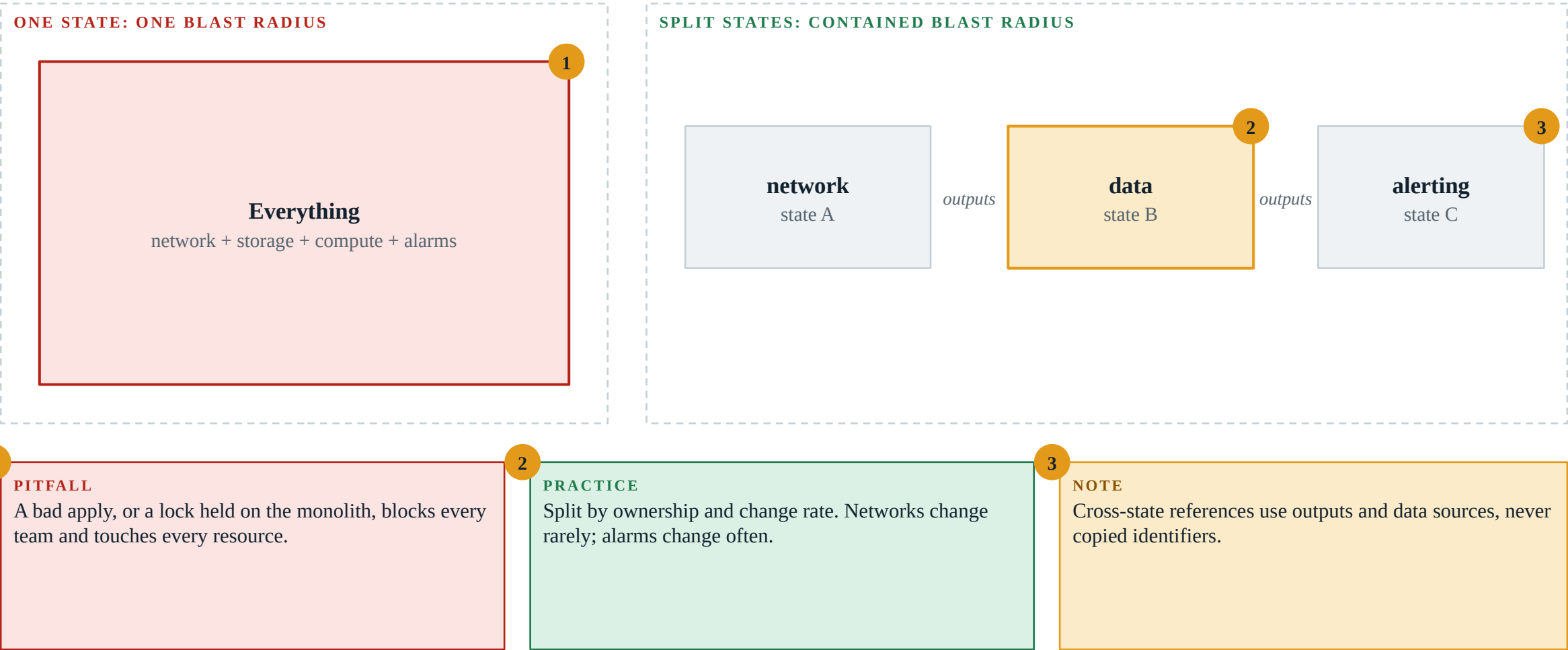
Locking stops two applies from corrupting state. Older setups used a DynamoDB table; check your version.

PRACTICE

Enable versioning on the state bucket. It is your undo button if state is ever damaged.

Blast radius: split state by ownership and change rate

A single monolithic state turns every apply into a company-wide event.



Calling a module

A pipeline is declared in ten lines. The module supplies the rest.

```
module "orders_pipeline" {  
  source = "app.terraform.io/meridian/data-pipeline/aws"  
  version = "~> 2.3"  
  
  name = "orders"  
  environment = "prod"  
  owner = "team-commerce-data"  
  schedule = "cron(0 2 * * ? *)"   
  freshness_slo_minutes = 240  
  alert_channel = var.pagerduty_service  
}
```

HCL

1

VERSION

Pin the module version. "~> 2.3" allows minor updates; use an exact version if you review each upgrade.

2

OWNER

Owner is a required input. Ownerless infrastructure is how orphans and surprise bills start.

3

SLO

The SLO input wires up the freshness alarm automatically. Monitoring ships with the pipeline.

Inside the module: safe defaults for stateful resources

Deletion protection and versioning on the raw bucket.

```
resource "aws_s3_bucket" "raw" {  
  bucket = "${var.name}-${var.environment}-raw"  
  
  lifecycle {  
    prevent_destroy = true  
  }  
}  
  
resource "aws_s3_bucket_versioning" "raw" {  
  bucket = aws_s3_bucket.raw.id  
  versioning_configuration {  
    status = "Enabled"  
  }  
}
```

HCL

1

NAMING

Name is derived from inputs, so it is stable. Changing it forces a replacement.

2

PROTECT

prevent_destroy makes any plan that would delete this bucket fail loudly.

3

UNDO

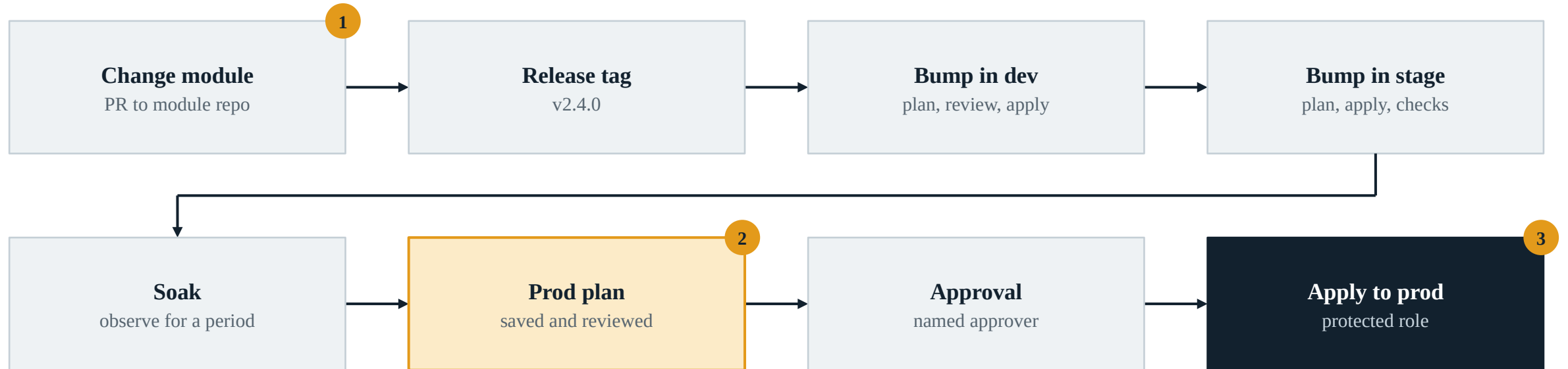
Versioning turns accidental overwrites and deletes into recoverable events.

NOTE

Give the pipeline's IAM role only this bucket and only the actions it needs. Least privilege belongs in the module too.

Promoting infrastructure through environments

Same module code everywhere. Only variables and versions differ.



1

NOTE

Every environment consumes the same module code. Differences live in variables.

2

PRACTICE

Promote by version: bump the module reference, dev then stage then prod.

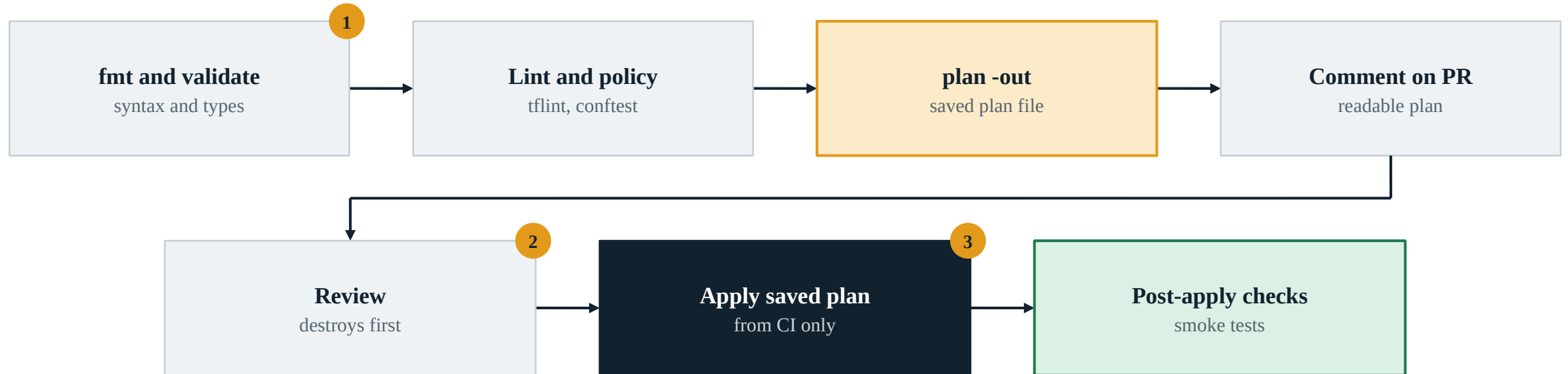
3

GUARDRAIL

Prod applies from CI with a saved plan and the protected apply role.

The change workflow: plan, review, apply

Humans review a plan. Automation applies exactly that plan.



1

NOTE

Cheap static checks fail fast, before anyone reads a plan.

2

PRACTICE

Look at destroys and replacements first, then IAM, then everything else.

3

STALE PLANS

Applying the saved plan means what was reviewed is what runs. If state moved, re-plan.

Reading a plan like a reviewer

Truncated plan output. Three lines matter most.

Plan output

```
# aws_s3_bucket.raw must be replaced
-/+ resource "aws_s3_bucket" "raw" {
  ~ bucket = "orders-prod-raw" -> "orders-prod-raw-v2"
    # forces replacement
}
```

```
# aws_iam_role.pipeline will be updated in-place
~ resource "aws_iam_role" "pipeline" {
  ~ max_session_duration = 3600 -> 7200
}
```

Plan: 1 to add, 1 to change, 1 to destroy.

1

REPLACE

-/+ means destroy, then create. On a bucket, that is data loss.

1

2

2

CAUSE

The comment names the argument that cannot change in place.

3

3

SUMMARY

Read the destroy count first. Any non-zero number needs an explanation.

Dangerous plan changes and their guards

What to look for, and what should already be protecting you.

Signal in the plan	Meaning	Data-platform risk	Guard
Replace on storage or a database	Destroy, then create	Data loss	prevent_destroy; deletion protection; versioning
Destroy count above zero	Something is being deleted	Lost topics, roles or schedules	Explicit approval for destroys
IAM policy widened	Broader permissions	Data exposure	Policy-as-code checks; second reviewer
count index shifts	Resources renumbered	Cascading replacement	for_each with stable keys
Provider or module upgrade in the plan	Behaviour changes	Surprise diffs	Upgrade separately from other changes
Sensitive value changes	Secret replaced or rotated	Broken authentication	Coordinate rotations deliberately

count versus for_each

Identify resources by a stable key, not by their position in a list.

HCL

```
# before: identified by list position
resource "aws_sns_topic" "t" {
  count = length(var.topics)
  name = var.topics[count.index]
}
```

```
# after: identified by key
resource "aws_sns_topic" "t" {
  for_each = toset(var.topics)
  name = each.key
}
```

1

1

FRAGILE

Remove "orders" from the front of the list and every later topic shifts index, so each is destroyed and recreated.

2

2

STABLE

With for_each, removing a key touches only that resource.

RULE

Migrating from count to for_each? Use moved blocks so the change is a rename, not a destroy.

Refactoring safely: moved, import, removed

Declarative blocks replace risky manual state surgery.

HCL

```
moved {  
  from = aws_s3_bucket.raw  
  to   = module.storage.aws_s3_bucket.raw  
}
```

```
import {  
  to = aws_iam_role.legacy_loader  
  id = "legacy-loader"  
}
```

```
removed {  
  from = aws_glue_job.old_job  
  lifecycle { destroy = false }  
}
```

1

MOVED

Renames or reorganises code without destroying the real resource.

1

2

IMPORT

Adopts an existing resource into state, and shows the result in a plan you can review.

2

3

REMOVED

Stops managing the resource but leaves it running.

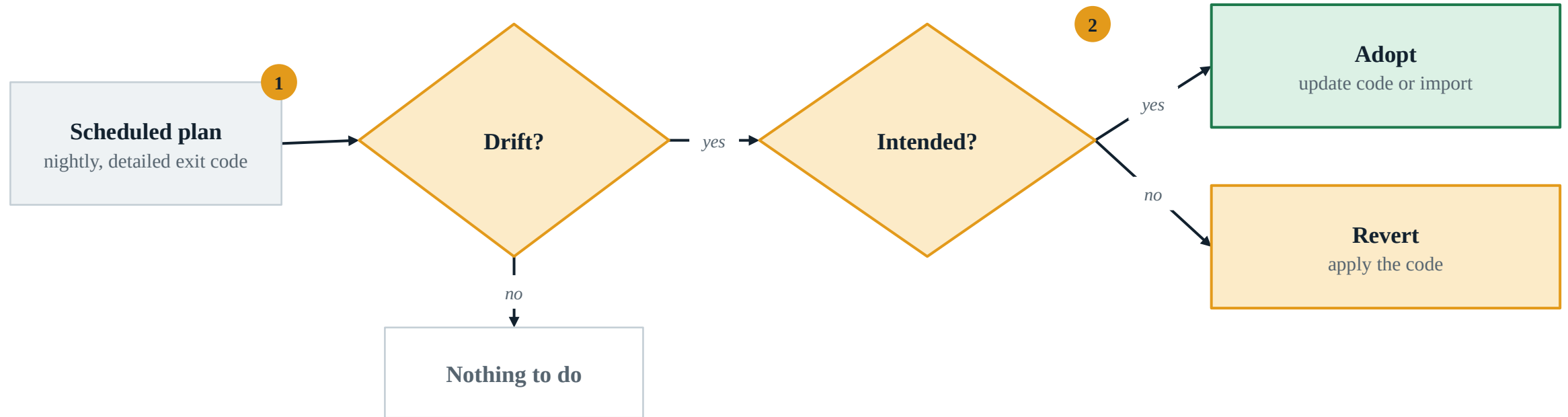
3

VERSIONS

These blocks arrived over several Terraform releases. Confirm your version supports each one before relying on it.

Drift: detect it, decide, and close the gap

Drift is any difference between the code and reality. Time is what makes it dangerous.



1

NOTE

Run a scheduled plan with `-detailed-exitcode`. Exit code 2 means changes are pending, which is your drift signal.

2

PRACTICE

Most drift is a well-meant console fix. Adopt it in code or revert it. Never leave it.

BREAK-GLASS

Allow console changes in an emergency, but require a ticket and a code follow-up within a day.

Policy as code: enforce standards in the plan

An OPA/Conftest policy that requires ownership tags on buckets.

Rego

```
package terraform.tags
import rego.v1

required := {"owner", "cost_center", "environment"}

deny contains msg if {
  rc := input.resource_changes[_]
  rc.type == "aws_s3_bucket"
  tags := object.get(rc.change.after, "tags", {})
  missing := required - {k | tags[k]}
  count(missing) > 0
  msg := sprintf("%s missing tags: %v", [rc.address, missing])
}
```

1

INPUT

The input is the plan as JSON, from terraform show -json on the saved plan.

1

2

LOGIC

Set subtraction finds which required tags are absent.

2

3

3

MESSAGE

The message appears in the CI failure, so authors know what to fix.

RULE

Tag policy is how cost allocation (Part 8) is enforced. No tags, no merge.

Testing infrastructure code

Native Terraform tests assert on plans without creating anything.

HCL

```
# tests/storage.tftest.hcl
variables {
  name = "orders"
  environment = "test"
}

run "bucket_blocks_public_access" {
  command = plan

  assert {
    condition = aws_s3_bucket_public_access_block.raw.block_public_acls
    error_message = "Public ACLs must be blocked"
  }
}
```

1

1

PLAN VS APPLY

command = plan is fast and free. command = apply builds real, short-lived resources.

2

2

ASSERT

Assert on the security and cost properties you care about, not on every attribute.

NOTE

For heavier integration tests, tools such as Terratest deploy into a sandbox account and check behaviour.

Secrets and state

Depth 4 begins. State is a sensitive artifact.

RISK**State can hold secrets**

- Passwords and keys may sit in plaintext.
- Anyone who can read state can read them.

CONTROL**Encrypt and restrict**

- Encrypted, versioned bucket.
- Narrow IAM on read and write.
- Never use local state for shared stacks.

CONTROL**Keep values out**

- Reference a secrets manager by identifier.
- Newer Terraform offers ephemeral and write-only values that stay out of state.

PRACTICE**Rotate on purpose**

- Make rotation a reviewed change.
- Avoid rotations that happen as a side effect of drift.

VERSIONS

Support for ephemeral resources and write-only arguments depends on your Terraform and provider versions. Check before designing around them.

Alarms as code: freshness

The pipeline module ships its own alarm, so monitoring cannot be forgotten.

HCL

```
resource "aws_cloudwatch_metric_alarm" "freshness" {  
  alarm_name = "gold-forecast-freshness"  
  namespace = "Meridian/Pipelines"  
  metric_name = "FreshnessLagMinutes"  
  dimensions = { table = "gold.forecast" }  
  statistic = "Maximum"  
  period = 300  
  evaluation_periods = 3  
  threshold = 240  
  comparison_operator = "GreaterThanThreshold"  
  treat_missing_data = "breaching"  
  alarm_actions = [var.pagerduty_topic_arn]  
}
```

1

2

3

1

DEBOUNCE

Three consecutive 5-minute periods: a sustained breach, not a blip.

2

THRESHOLD

Threshold comes from the SLO input (slide 82), not a hand-tuned number.

3

MISSING DATA

If the metric stops arriving, treat that as a failure. Silence is a symptom.

Cost guardrails as code

Tags applied everywhere, and storage that tiers and expires by default.

```
provider "aws" {  
  default_tags {  
    tags = {  
      owner = var.owner  
      cost_center = var.cost_center  
      environment = var.environment  
    }  
  }  
}  
  
resource "aws_s3_bucket_lifecycle_configuration" "raw" {  
  bucket = aws_s3_bucket.raw.id  
  rule {  
    id = "tier-and-expire"  
    status = "Enabled"  
    filter {}  
    transition {  
      days = 30  
      storage_class = "STANDARD_IA"  
    }  
    expiration { days = 730 }  
  }  
}
```

HCL**1****1****TAGS**

default_tags stamps every taggable resource, so allocation needs no per-resource effort.

2**2****TIER**

Cold data moves to a cheaper storage class after 30 days.

3**3****EXPIRE**

Expiry must match retention policy and legal duty. Never guess.

Gate 4: Terraform readiness

Evidence that infrastructure changes are safe to automate.

Structure

- ☐ One owner per object; seams documented
- ☐ State split by ownership and change rate
- ☐ Remote, encrypted, versioned, locked state
- ☐ Modules versioned; prod pins exact versions

Change safety

- ☐ Only CI applies; plans are saved and reviewed
- ☐ Destroys and replacements need explicit approval
- ☐ Stateful resources have deletion protection
- ☐ Scheduled drift detection routes to on-call

Governance

- ☐ Policy as code blocks untagged resources
- ☐ Security properties covered by tests
- ☐ Secrets kept out of code and state where possible
- ☐ Alarms and budgets ship inside the modules

RULE

If a resource can be changed in the console without anyone noticing, it is not under control.

06

Monitoring and alerting

Measure what consumers feel, and wake a human only when it matters.

SLIDES 97–120 · DEPTH 4

IN THIS PART

- The observability pillars for data and ML
- SLIs, SLOs, freshness and volume monitoring
- Statistical thresholds and drift metrics
- Lineage-aware monitoring and dashboards
- Alert philosophy, severity and burn-rate alerting
- Alert rules as code, routing and grouping
- Alert fatigue and Gate 5

Six pillars of data and ML observability

Each answers a different question. Together they make failures visible.

PILLAR**1****Freshness**

- Did the data arrive on time?
- Time since last successful update, per table.

PILLAR**2****Volume**

- Did the expected amount arrive?
- Row counts and bytes against the usual pattern.

PILLAR**3****Schema**

- Did the structure change?
- Columns added, removed or retyped.

PILLAR**4****Distribution**

- Do the values look normal?
- Nulls, ranges, cardinality, quantiles.

PILLAR**5****Lineage**

- Who is affected, and where did it start?
- The upstream and downstream graph.

PILLAR**6****Model behaviour**

- Are predictions healthy?
- Input drift, prediction drift, delayed accuracy.

PITFALL

Task success is the least sufficient signal. A green pipeline can still carry wrong data.

A signal map: what to measure at each layer

Alert near the consumer. Use lower layers to explain, not to announce.

Layer	What you measure	Example metrics	Alert?
Infrastructure	Compute, storage, network	CPU, memory, spill, disk, queue depth	Rarely; use for diagnosis
Orchestration	Task and DAG outcomes	Success rate, duration, retries, start lag	On sustained failure
Data	Freshness, volume, schema, quality	Landing time, row count, null rate	Yes, linked to SLOs
Model	Inputs, outputs, accuracy	PSI, prediction mean, WAPE with delay	Mostly tickets
Business	Outcomes consumers care about	Forecast coverage, revenue variance	Yes, when tied to an SLO
Cost	Spend and unit cost	Daily spend, cost per 1,000 forecasts	On anomalies

GOLDEN SIGNALS

The classic four translate directly: latency is freshness lag, traffic is volume, errors are failures and invalid rows, saturation is backlog.

Instrumentation: metrics tell you that, logs tell you which

One function emits both, so the two views always agree.

Python

```
import json, logging, time
from opentelemetry import metrics

log = logging.getLogger("pipeline")
meter = metrics.get_meter("meridian.pipeline")
rows_written = meter.create_counter("rows_written")
run_seconds = meter.create_histogram("run_seconds")

def finish(run_id, table, day, rows, started):
    secs = time.time() - started
    attrs = {"table": table}
    rows_written.add(rows, attrs)
    run_seconds.record(secs, attrs)
    log.info(json.dumps({"event": "run_done",
                        "run_id": run_id, "day": day,
                        "table": table, "rows": rows}))
```

1

CORRELATE

The same run_id appears in the run ledger (Part 2), so every signal joins up.

1

2

CARDINALITY

Metric attributes must be low-cardinality. Never label by run_id or user id.

2

3

STRUCTURED LOG

Logs carry the high-cardinality detail: run_id, partition, counts. Search here to find which run.

3

SLIs and SLOs for data products

An SLI is a ratio of good events to total events. An SLO is its target.

SLI	Good event	Total events	Example SLO
Freshness	A daily run lands before its deadline	Days in the window	99% of days over 30 days
Completeness	A partition has every expected source	Partitions	99.9% of partitions
Validity	A row passes all critical quality tests	Rows	99.95% of rows
Accuracy	A day reconciles to source within tolerance	Days	100% of days within 0.1%
Availability	A forecast lookup succeeds	Lookups	99.9% of lookups

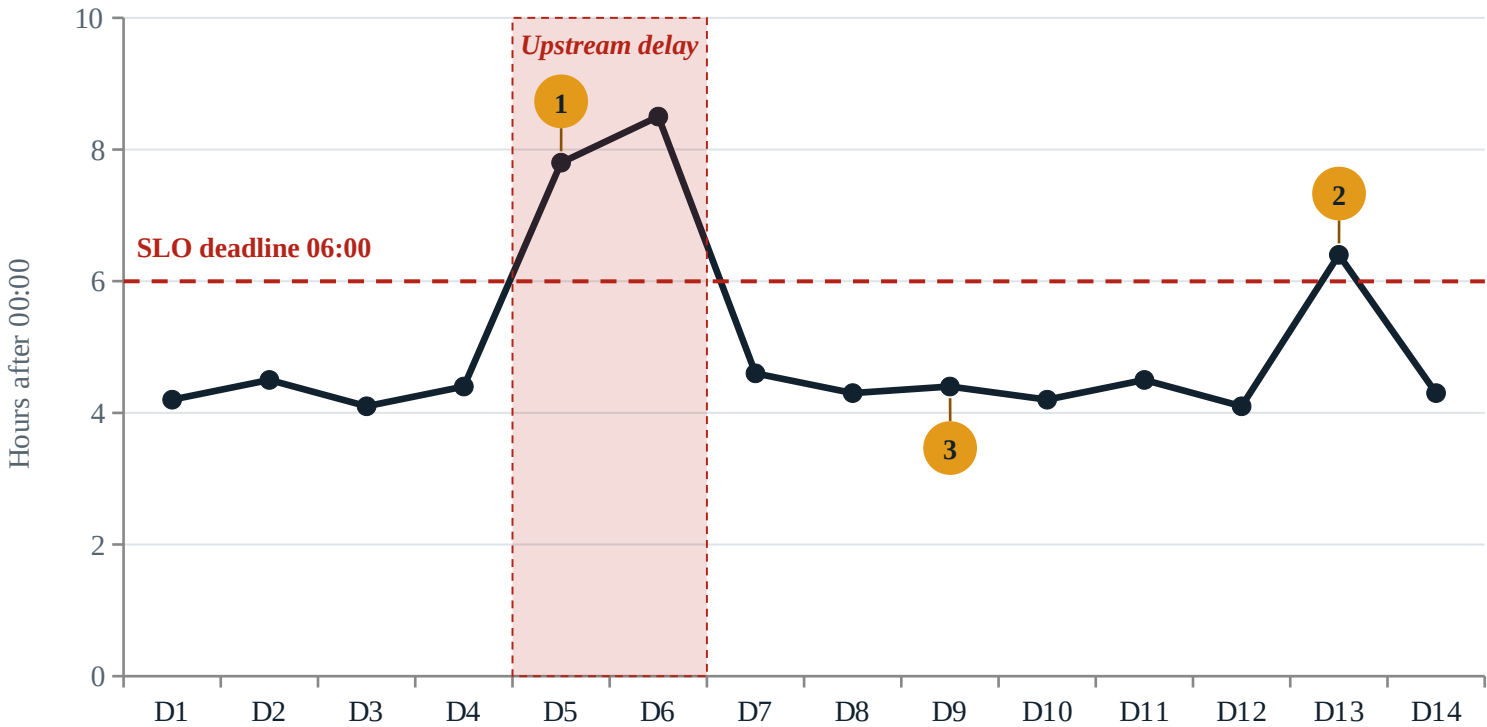
DEFINITION

Error budget = 1 minus the SLO. A 99% SLO over 30 days allows 1% of events to be bad.

Freshness against a deadline

Deadlines are binary. Late by a minute is late.

Figure 5. Landing time of gold.forecast, hours after midnight, over 14 days (illustrative)



1

PITFALL

Days 5 and 6: an upstream delay causes two consecutive misses.

2

PITFALL

Day 13 lands at 06:24. Only 24 minutes late, and still a miss.

3

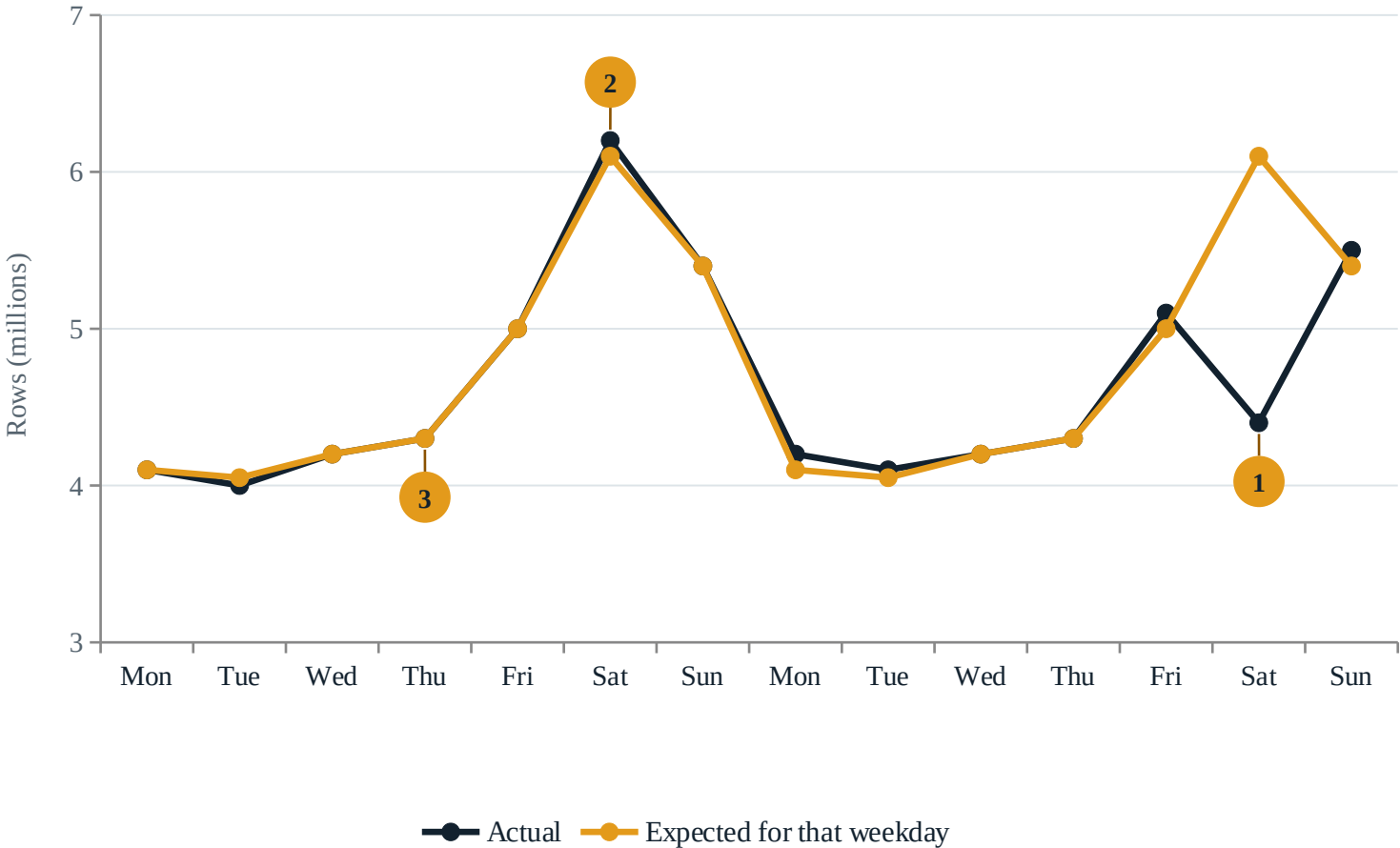
BUDGET

Three misses in 14 days. A 99% SLO allows about 0.3 misses per 30 days: the budget is gone.

Volume monitoring needs seasonality

A fixed floor would miss this anomaly. A weekday-aware baseline catches it.

Figure 6. Daily order rows in millions versus expected for the same weekday (illustrative)



1

PITFALL

Saturday is 28% below expected. A fixed floor of 4M would not fire.

2

SEASONALITY

Saturdays are normally the weekly peak.

3

PRACTICE

Weekday-aware baselines cut false alarms without missing real drops.

Robust thresholds: median and MAD

Baselines that shrug off yesterday's incident.

Python

```
import numpy as np

def robust_z(history, today):
    med = np.median(history)
    mad = np.median(np.abs(history - med))
    scale = 1.4826 * mad or 1.0
    return (today - med) / scale

# same weekday, last 8 weeks
z = robust_z(rows_same_weekday[-8:], rows_today)
if abs(z) > 4:
    alert("volume_anomaly", z=z)
```

1

ROBUST

Median and MAD resist outliers, so an old incident does not distort the baseline.

1

2

2

SCALE

The 1.4826 factor makes MAD comparable to a standard deviation for normal data. A zero MAD needs a floor.

3

3

THRESHOLD

Illustrative cut-off. Tune it against historical false positives and misses.

PRACTICE

Statistical alerts are candidates, not verdicts. Route them to tickets until their precision is proven.

Distribution drift: the population stability index

Compare a reference sample with recent data, one feature at a time.

Python

```
def psi(expected, actual, bins=10):  
    q = np.linspace(0, 1, bins + 1)  
    cuts = np.quantile(expected, q)  
    cuts[0], cuts[-1] = -np.inf, np.inf  
    e = np.histogram(expected, cuts)[0] / len(expected)  
    a = np.histogram(actual, cuts)[0] / len(actual)  
    e = np.clip(e, 1e-6, None)  
    a = np.clip(a, 1e-6, None)  
    return float(np.sum((a - e) * np.log(a / e)))
```

1

BINS

Bin edges come from the reference distribution's quantiles, so bins start equally populated.

2

CLIP

Clip to avoid log of zero on empty bins.

3

SCORE

The index sums how far each bin's share has moved, weighted by log-ratio.

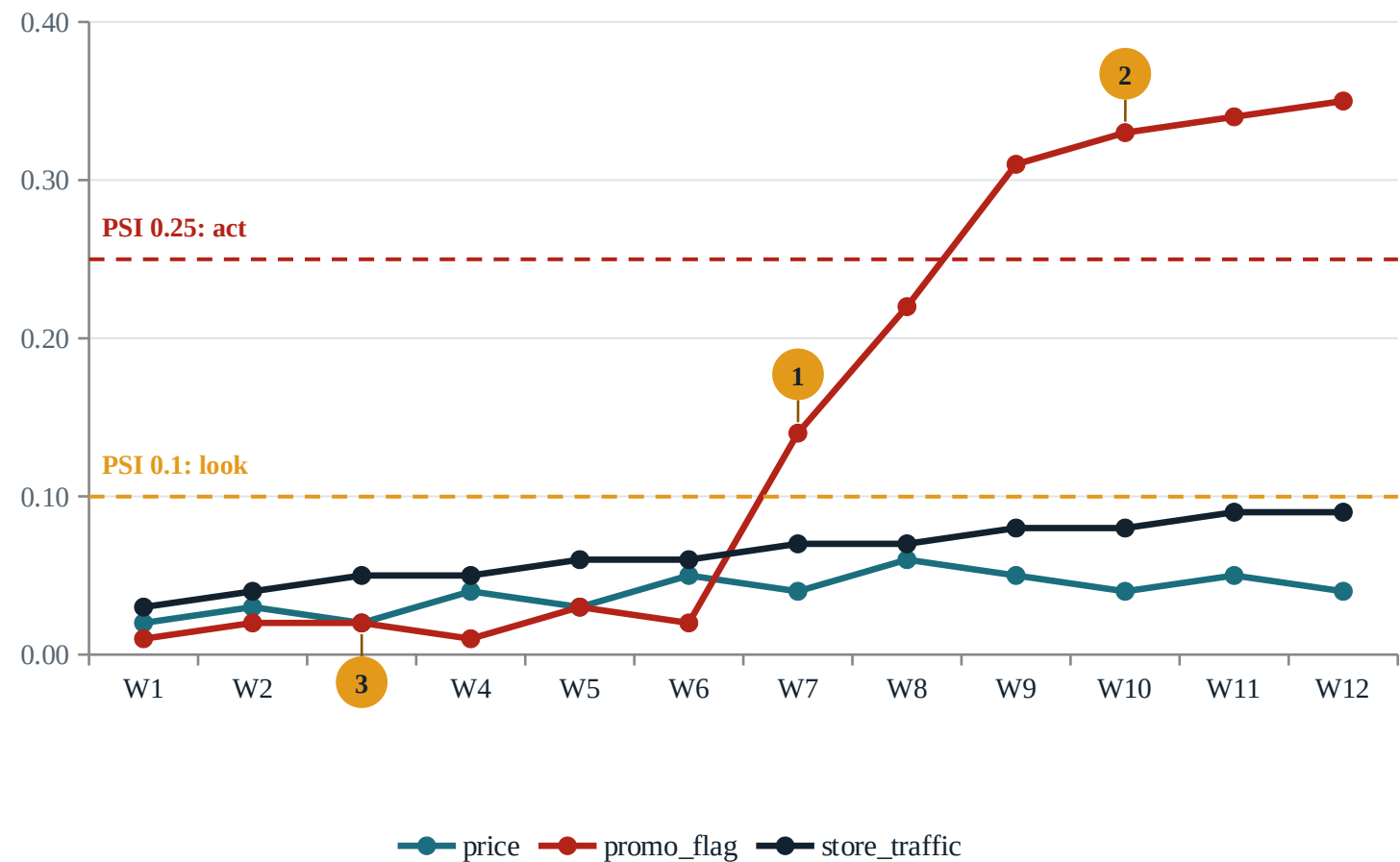
RULES OF THUMB

Below 0.1 is usually read as stable, 0.1 to 0.25 as moderate shift, above 0.25 as major. These conventions come from credit scoring, so calibrate them on your own data.

Feature drift over twelve weeks

One feature changed meaning upstream. The others just wobble.

Figure 7. PSI per feature against the training reference, weekly (illustrative)



1

PITFALL
Week 7: promo_flag crosses 0.1 after an upstream coding change.

2

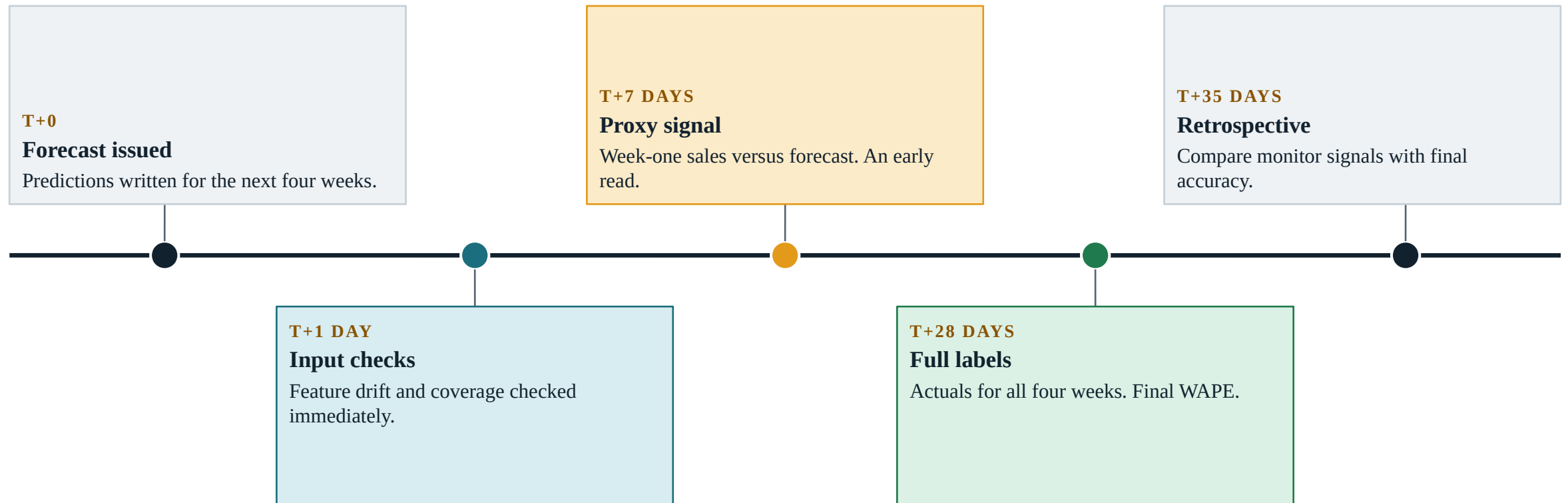
PITFALL
Sustained above 0.25. The feature no longer means what the model learned.

3

PRACTICE
Other features wander within normal noise. No action.

Accuracy arrives late: monitor inputs and proxies meanwhile

Ground truth for a four-week forecast takes about four weeks.

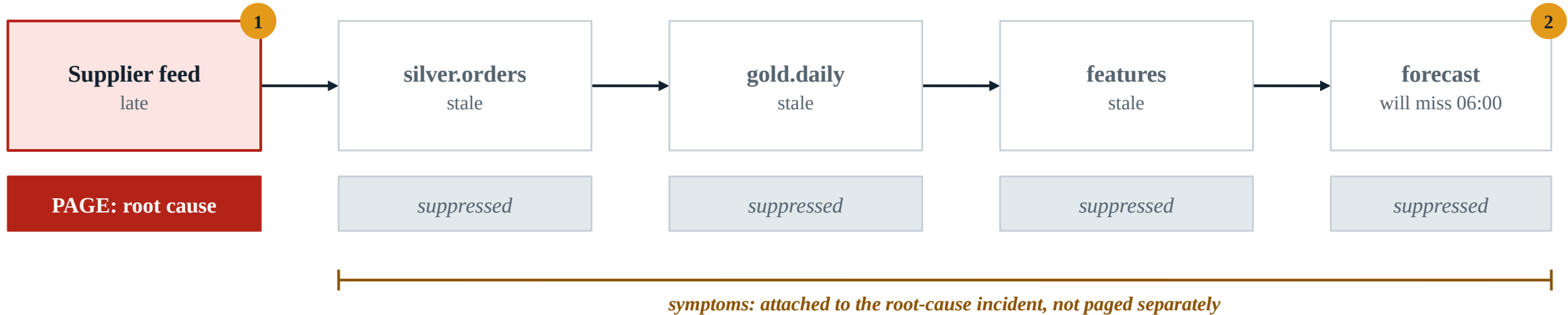


RULE

Between T+0 and T+28 you fly without ground truth. Monitor inputs and proxies, then close the loop when labels land.

Lineage-aware alerting: one root cause, one page

When a feed is late, everything downstream is late too. Say so once.



1 **PITFALL**
Without lineage, one late feed pages four teams for four symptoms. Each wastes time rediscovering the same cause.

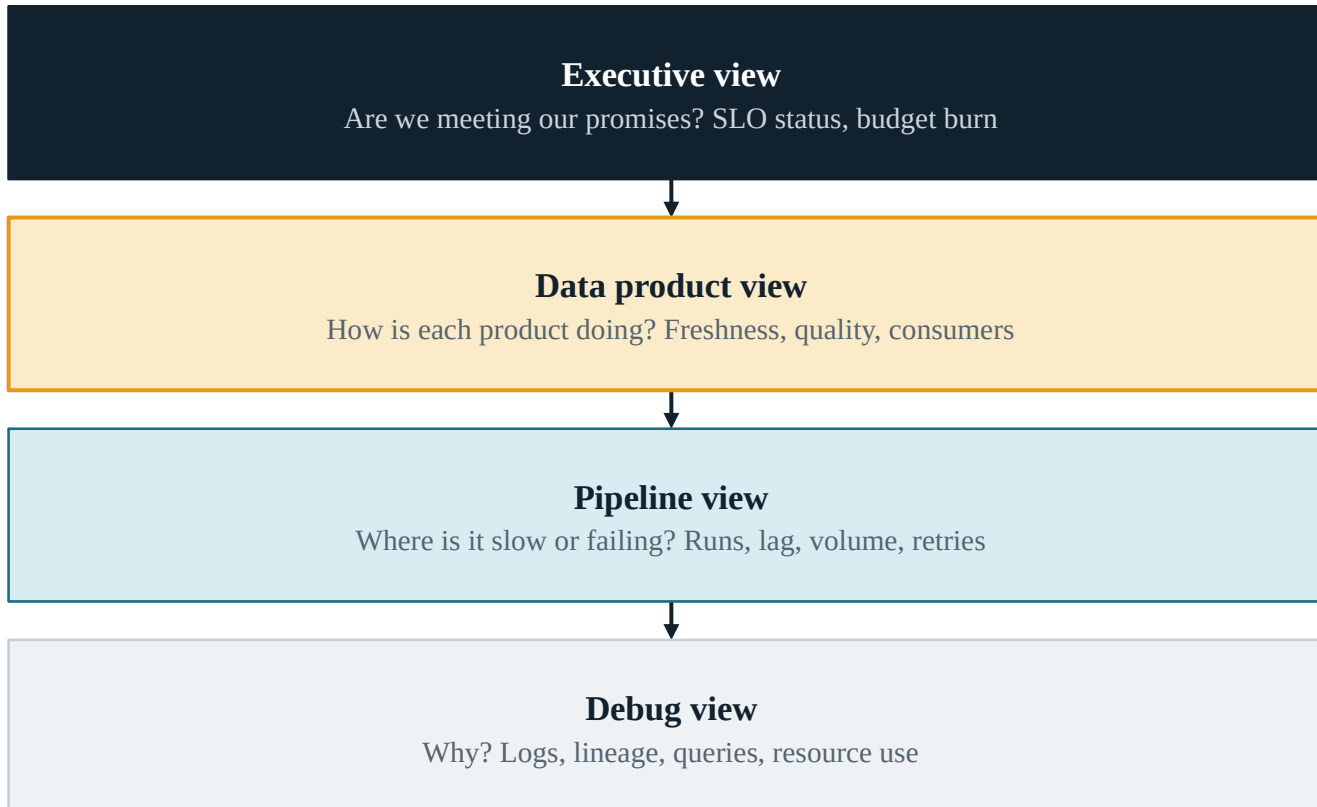
2 **PRACTICE**
With lineage, the alert points to the root cause and mutes descendants while it is open. The forecast owner is informed, not paged.

START SMALL

This needs trustworthy lineage. Begin with the critical path to your SLOs and grow outward.

Dashboards in layers: each answers one question

Nobody should need to open a query editor to learn whether a promise is being kept.



1

PRACTICE

Every panel links to the layer below. A red tile on top takes two clicks to a log line.

2

NOTE

Put SLO status on a single screen with the error budget remaining and burn rate.

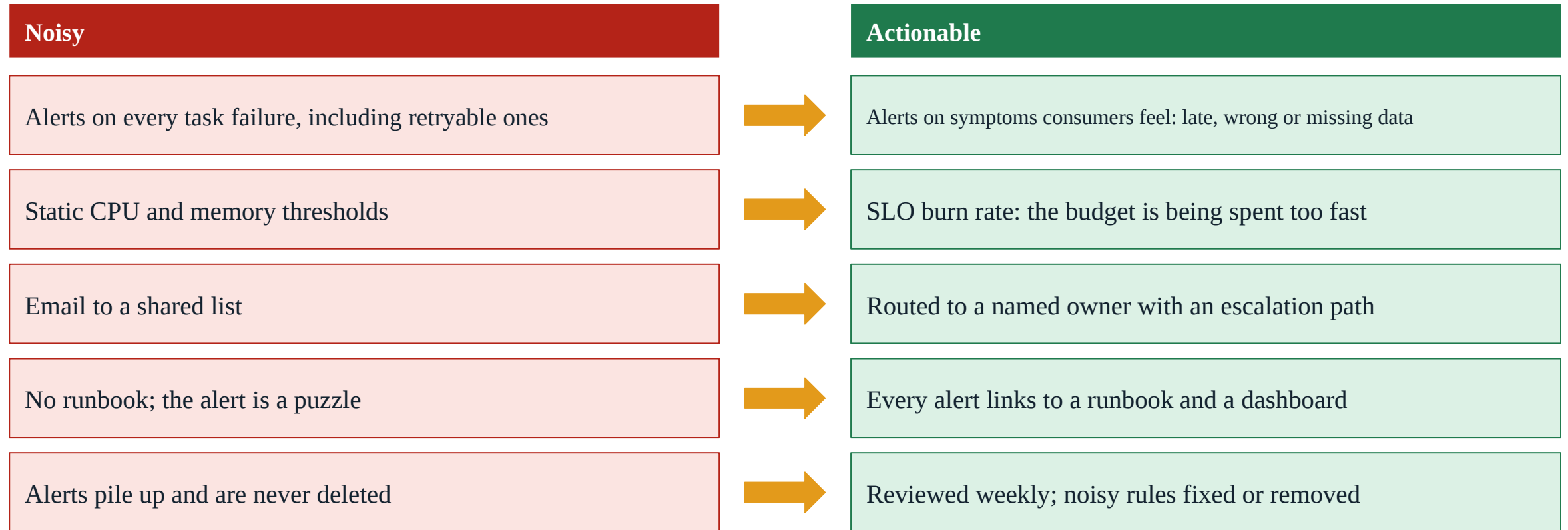
3

DASHBOARD SPRAWL

Dashboards nobody opens are noise. Retire them, and keep the layers in version control.

Noisy alerting versus actionable alerting

Depth 4 continues. The change is mostly in what you choose not to alert on.

**RULE**

Every page must be urgent, actionable and real. Miss one of the three and it is a ticket or a dashboard.

The severity ladder

Decide the channel by the required response, not by the mood of the author.

Level	Criteria	Response	Channel	Example
Page	Consumer impact now or soon; a human must act	Minutes, any hour	Phone and push	Forecast will miss 06:00
Ticket	Real problem, no immediate impact	Within working days	Queue	Feature drift above 0.1 for a week
Notify	Informational; may need attention	Best effort	Chat channel	Backfill finished
Dashboard	Context for diagnosis	On demand	Dashboards	Cluster utilisation

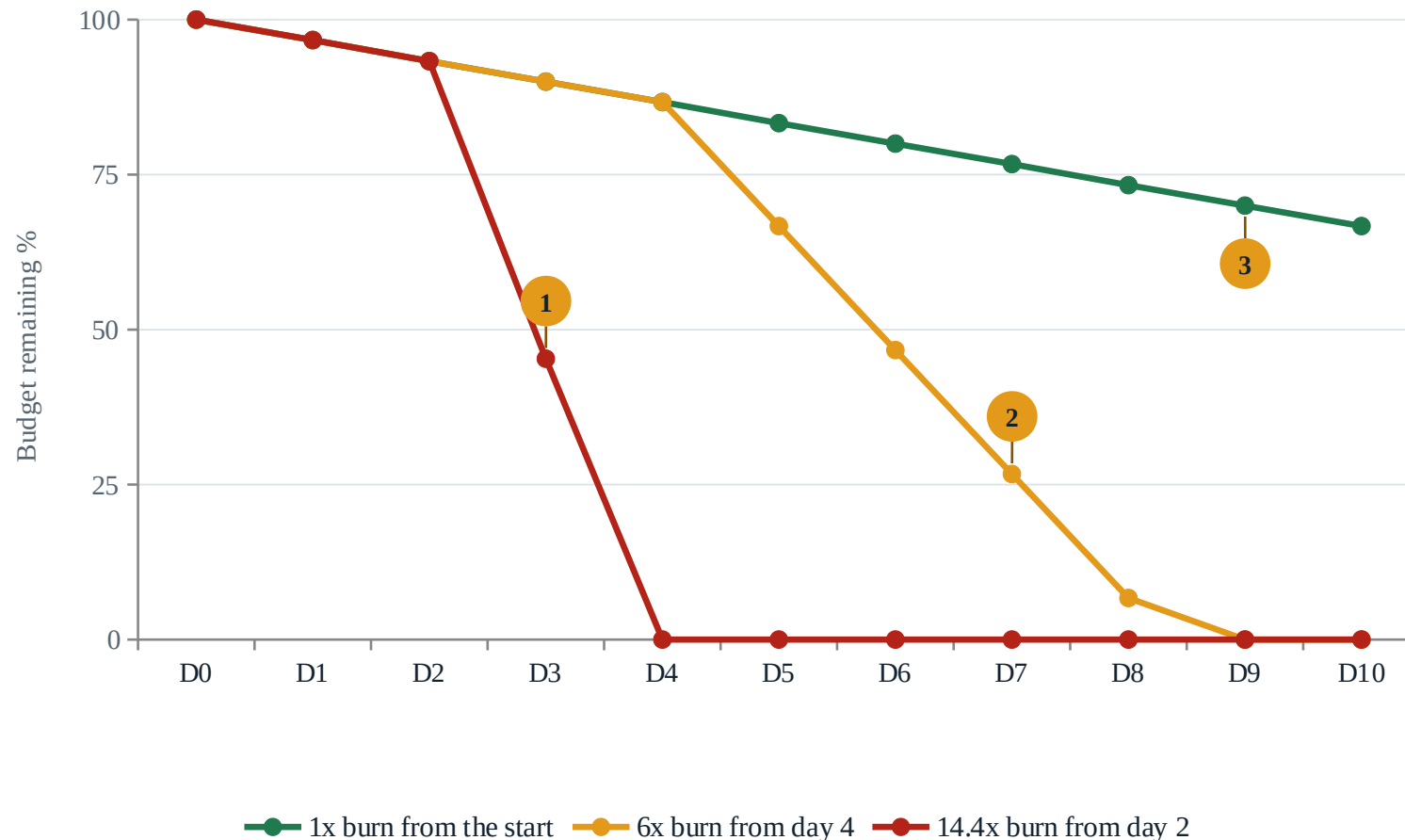
PRACTICE

If nobody would get out of bed for it, it must not page.

Burn rate: how fast the error budget disappears

One number tells you whether to page, ticket or relax.

Figure 8. Error budget remaining over ten days at three burn rates, 30-day SLO (illustrative)



1

PITFALL

At 14.4x, 2% of a 30-day budget vanishes every hour. Page at once.

2

NOTE

At 6x, 5% is gone in six hours. Page, with a little more time to think.

3

PRACTICE

At 1x the budget lasts exactly the window. A ticket, not a page.

Multi-window, multi-burn-rate alerts

A widely used pattern for a 30-day SLO. Each row needs both windows to be breached.

Severity	Burn rate	Long window	Short window	Budget spent	Budget gone in
Page	14.4	1 hour	5 minutes	2%	about 2 days
Page	6	6 hours	30 minutes	5%	5 days
Ticket	1	3 days	6 hours	10%	30 days

WHY TWO WINDOWS

The long window shows the burn is sustained. The short window shows it is still happening, so the alert clears soon after you fix the problem.

Alert rules as code

Prometheus-style rules. Fast burn on validity, plus a heartbeat check on the monitors.

YAML

```
groups:
- name: forecast-slo
  rules:
  - alert: ValidityFastBurn
    expr: |
      slo:validity_error_ratio:rate1h > (14.4 * 0.0005)
      and
      slo:validity_error_ratio:rate5m > (14.4 * 0.0005)
    labels: {severity: page, team: commerce-data}
    annotations:
      runbook_url: https://runbooks.example/validity
  - alert: MonitoringBlind
    expr: absent_over_time(pipeline_heartbeat_total[15m])
    labels: {severity: page}
```

1

THRESHOLD

99.95% validity leaves a 0.05% budget, so the fast-burn threshold is 14.4 times 0.0005.

1

2

2

TWO WINDOWS

Both windows must exceed the threshold: sustained and ongoing.

3

MONITOR THE MONITORS

A dead-man's switch: if the heartbeat metric disappears, the monitoring itself is broken. Page.

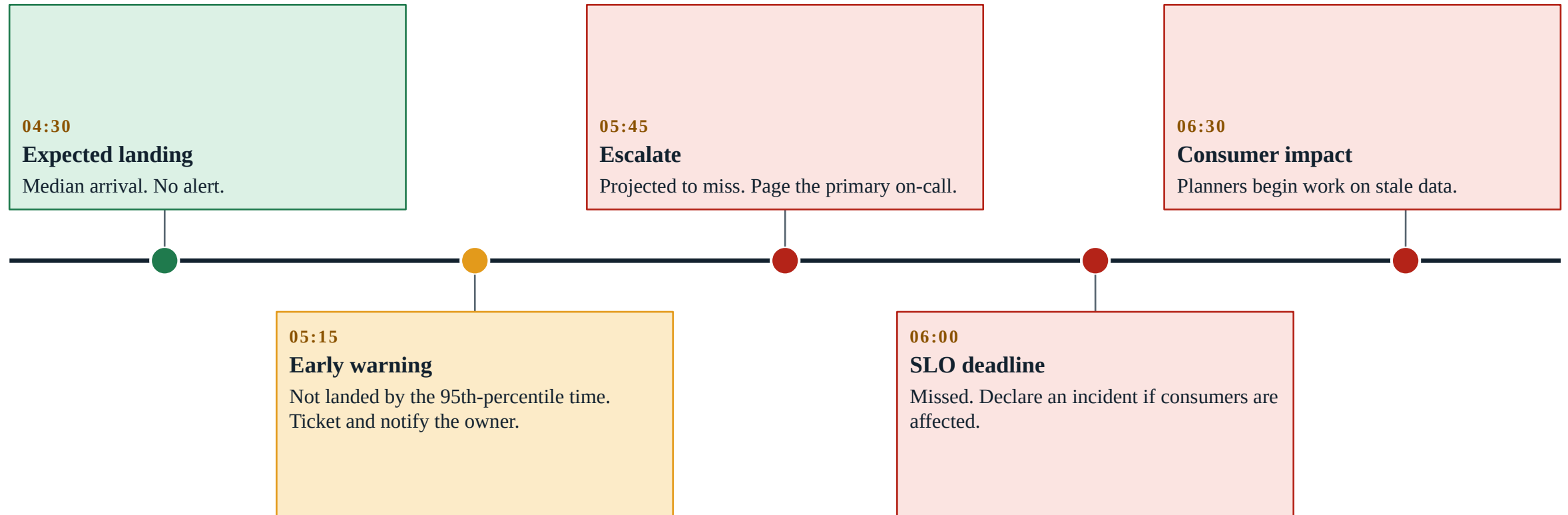
3

NOTE

Burn-rate windows suit high-event SLIs such as rows and requests. A once-a-day pipeline needs deadline alerts instead.

Deadline alerts for a daily pipeline

Alert on projected misses, before the SLO is actually broken.



RULE

Daily jobs have too few events for burn-rate windows. Alert on the deadline and on projected misses. The times here are illustrative.

Anatomy of a good alert

A page should let a half-asleep engineer act in one minute.

1

PRACTICE

The symptom in plain words, not a metric name.

2

PRACTICE

When it started and how bad: time, burn rate, threshold.

3

PRACTICE

Where to look: dashboard, runbook, and the last change.

[PAGE] Forecast freshness: projected to miss 06:00

Impact: planners will see yesterday's forecast after 06:00

Since 05:45 · expected 04:30 · SLO: 99% of days by 06:00

Owner: commerce-data on-call · escalates in 15 minutes

Dashboard · Runbook · Last deploy 05:31 (pull request 1234)

First step: check whether raw.orders landed

4

NOTE

Impact in consumer terms: who is affected and how.

5

NOTE

Who owns it and where it escalates.

6

NOTE

The most likely first action.

Routing, grouping and inhibition

Alertmanager-style configuration that turns raw alerts into a small number of notifications.

YAML

```
route:
  receiver: default-ticket
  group_by: [alertname, team]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - matchers: [severity="page"]
      receiver: oncall-pager

inhibit_rules:
  - source_matchers: [alertname="UpstreamLate"]
    target_matchers: [severity="page"]
    equal: [pipeline]
```

1

1

GROUP

group_by merges related alerts into one notification, so a storm becomes a single message.

2

2

REPEAT

Repeat interval limits nagging for an alert that is already known.

3

3

INHIBIT

While UpstreamLate is firing, mute downstream pages for the same pipeline.

PITFALL

Routing by label needs consistent labels. Enforce team and severity labels on every rule, ideally in CI.

Alert fatigue: measure it

Illustrative targets. What matters is that you track them and act on the trend.

≤ 2

Pages per on-call shift
more than this erodes response quality

> 80%

Pages that needed action
the rest are candidates to tune or delete

< 5 min

Median time to acknowledge
a symptom of trust in the pager

< 10%

Alerts that auto-resolve
flapping or noisy rules

Sign of fatigue	Response
Pages that resolve on their own	Add debounce, or delete the alert
Acknowledged with no action taken	Downgrade to a ticket
The same alert every week	Fix the root cause or the threshold
Engineers dread the pager	Cut scope; review load; fix the top three sources

The weekly alert review

Thirty minutes a week beats a quarterly alert amnesty.

1	Collect	Pull last week's pages, tickets and silences.	EVIDENCE <i>Alert history grouped by rule and owner</i>
2	Classify	Actionable, informational, false positive or duplicate.	EVIDENCE <i>Every page labelled in the review notes</i>
3	Fix or delete	Tune the threshold, add debounce, downgrade or remove.	EVIDENCE <i>Each noisy rule gets an owner and a date</i>
4	Test	Unit-test rules; inject a synthetic failure in staging.	EVIDENCE <i>Rule tests pass; drill fires the right page</i>
5	Heartbeat	Confirm the monitors themselves are alive.	EVIDENCE <i>Dead-man's switch received its test signal</i>
6	Record	Log rule changes and re-baseline the SLOs if needed.	EVIDENCE <i>A changelog entry per rule change</i>
PRACTICE Rotate the reviewer. Fresh eyes spot the alerts the regulars have learned to ignore.			

Gate 5: monitoring and alerting readiness

What must be true before a data product is on-call ready.

Measure

- ☐ SLIs and SLOs defined per data product
- ☐ Freshness, volume and schema monitored
- ☐ Drift monitored with reviewed thresholds
- ☐ Dashboards layered with drill-down links

Alert

- ☐ Pages tied to SLO burn or deadlines
- ☐ Every alert has an owner, runbook and dashboard
- ☐ Routing, grouping and inhibition configured
- ☐ A heartbeat catches silent monitors

Sustain

- ☐ Weekly review with recorded changes
- ☐ Alert rules tested like code
- ☐ Pages per shift and actionability tracked
- ☐ Lineage suppresses downstream noise

RULE

A page without a runbook is a puzzle. A runbook without a page is a wiki. You need both.

07

Incident response

When the page arrives: contain, communicate, recover, and learn.

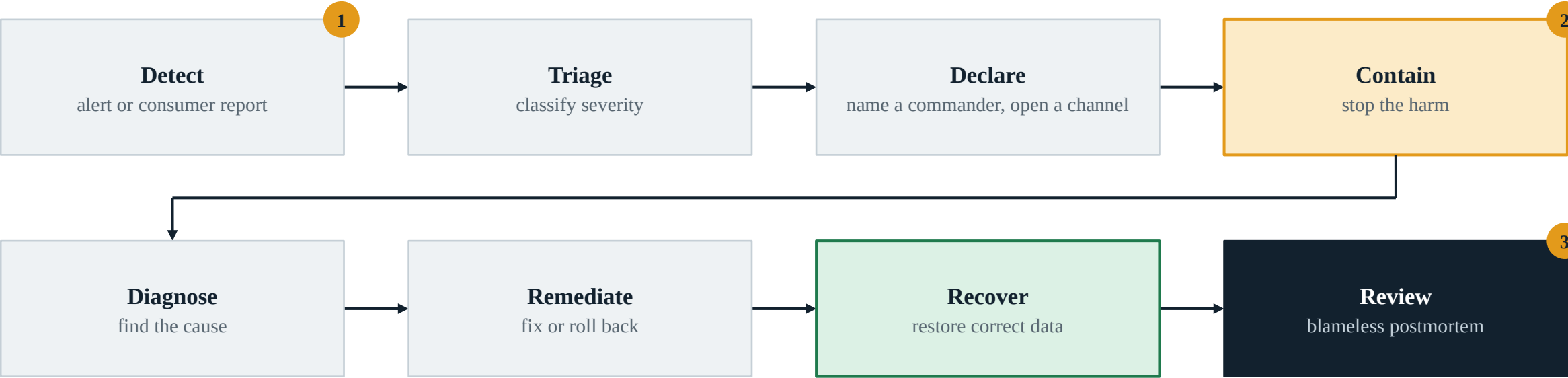
SLIDES 121–134 · DEPTH 4–5

IN THIS PART

- The incident lifecycle and severity classes
- Roles: commander, operations, communications, scribe
- Containment patterns for bad data
- Status updates and communication cadence
- Blameless postmortems and causal chains
- Playbooks for common data failures
- Incident metrics and Gate 6

The incident lifecycle

Eight stages. The order matters: contain before you diagnose.



1

DETECTION

Detection should come from monitors, not consumers. Track how often it does not.

2

PRACTICE

Stop the bleeding first. Restoring safe service is not the same as finding the cause.

3

NOTE

The review is where an incident pays for itself. Skipping it wastes the lesson.

Severity classes for data incidents

Classify by the decisions and obligations affected, not by whether the pipeline is green.

Class	Criteria	Response	Communication
SEV1	Wrong or missing data in a finance, regulatory or customer-facing output, or a platform-wide outage	Immediate; commander named; all hands	Leadership and consumers within 30 minutes
SEV2	An SLO is breached or about to be, and a key consumer is affected	Within 15 minutes; on-call and owner	Consumers notified within an hour
SEV3	Degraded service with a workaround; no SLO breach	Working hours	Update in the team channel
SEV4	Minor or cosmetic; no consumer impact	Backlog	None required

PITFALL

A pipeline that succeeds while producing wrong revenue is SEV1 in effect. Green status never lowers severity.

Four roles, one voice each

Small teams may combine roles, but each job still needs an owner.

ROLE 1

Incident commander

- Decides, delegates, keeps time.
- Does not debug.

ROLE 2

Operations lead

- Investigates and applies fixes.
- Reports findings to the commander.

ROLE 3

Communications lead

- Sole voice to consumers and leadership.
- Sends updates on the agreed cadence.

ROLE 4

Scribe

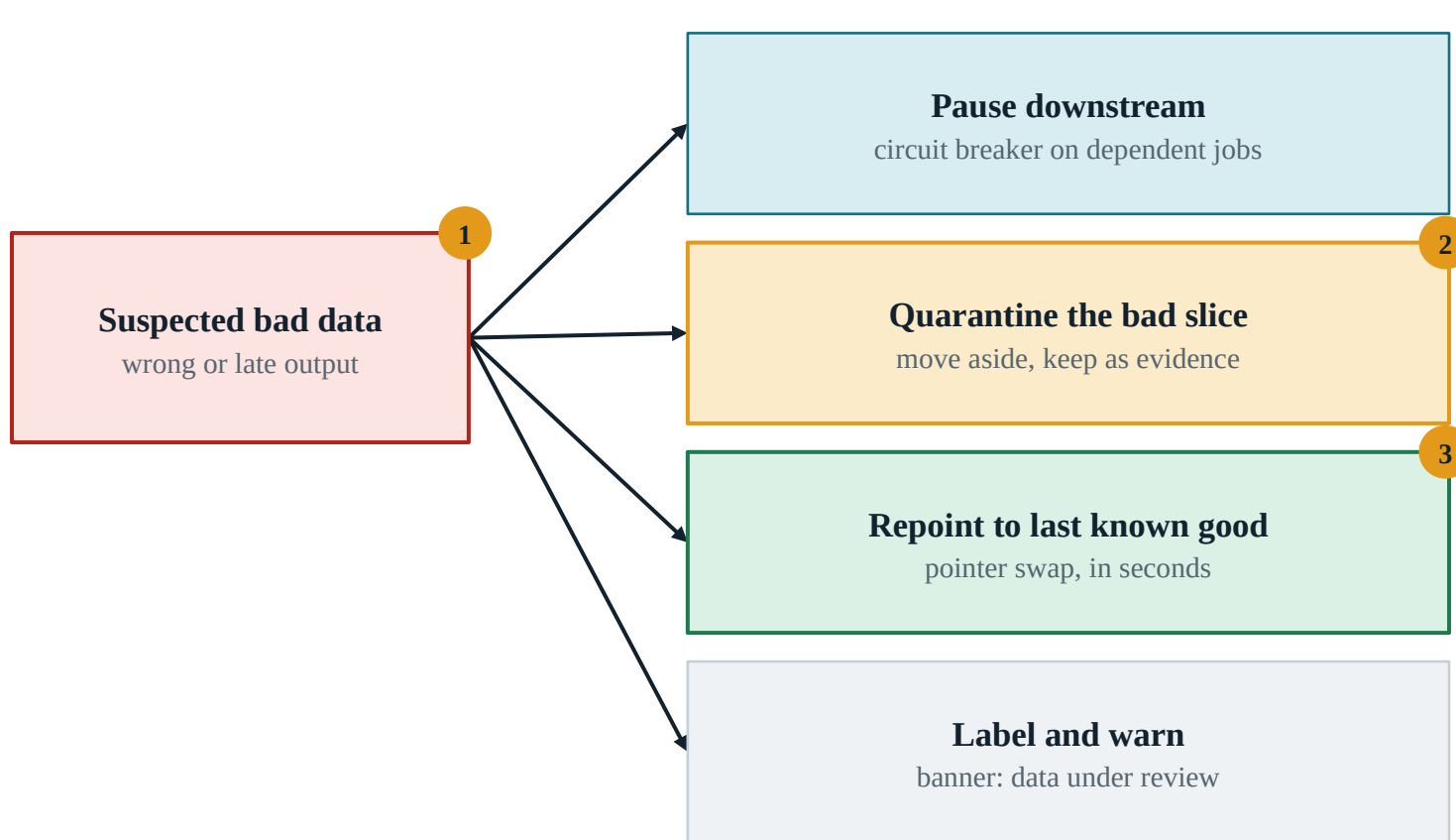
- Records actions, decisions and times.
- Feeds the postmortem timeline.

RULE

On a SEV1 never combine commander and operations lead. The person fixing cannot also be steering.

Containment options for bad data

Choose the least destructive action that stops consumers acting on wrong numbers.



1

NOTE

Contain first, diagnose second. You can investigate calmly once consumers are protected.

2

DO NOT DELETE

Quarantine, never destroy. You will need the bad data as evidence.

3

PRACTICE

Repointing is the cheapest rollback, and it needs the pointer design from Parts 3 and 4.

Containment by pointer: seconds, not hours

Consumers read through a view that points at a published version.

-- consumers read a view, never the raw table

```
CREATE VIEW serving.forecast AS
```

```
SELECT f.*
```

```
FROM gold.forecast_versions f
```

```
JOIN ops.published p
```

```
  ON f.version_id = p.version_id
```

```
WHERE p.dataset = 'forecast';
```

-- containment: repoint to the last known good

```
UPDATE ops.published
```

```
SET version_id = :last_good, note = :incident_id
```

```
WHERE dataset = 'forecast';
```

SQL

1

1

VIEW

Indirection is designed in advance. You cannot add it mid-incident.

2

2

SWAP

One row changes, so the action is fast, atomic and easy to reverse.

3

3

TRACE

Record the incident id so the change is traceable and gets undone deliberately.

PITFALL

Repointing hides the bad data from consumers but does not fix it. Track the follow-up work in the incident.

Communication cadence

People forgive bad news. They do not forgive silence.

**NOTE**

Cadences are illustrative. Agree them per severity class in advance, and give consumers one place to look.

A status update template

The same six fields every time, so readers know where to look.

Template

[SEV2] Forecast late: update 2
Status: Investigating | Identified | Mitigating | Resolved
Impact: planners see yesterday's forecast; stores unaffected
Since: 05:31 UTC (declared 05:52)
Known: supplier feed delayed; downstream tables stale
Doing: serving last known good; waiting on supplier
Next update: 07:00, or sooner if this changes
Commander: on-call lead

1

2

3

1

IMPACT

Impact is written in consumer terms: who is affected and how.

2

KNOWN

Separate what you know from what you suspect. Never guess in public.

3

PROMISE

Commit to the next update time, and keep it.

Blameless does not mean accountability-free

It means asking why the system allowed the mistake, not who made it.

Blaming language		Blameless language
An engineer pushed a bad change	➔	The change passed review and CI, which did not test this case
They should have been more careful	➔	What made the wrong action easy and the right action hard?
Root cause: human error	➔	Contributing factors: process, tooling and missing signals
Fix: remind everyone to double-check	➔	Fix: add a guardrail, a test or an alert
Discussed privately, then forgotten	➔	Published so other teams learn from it

RULE

People are accountable for follow-through on action items. Nobody is punished for being the one holding the keyboard.

The postmortem template

Seven sections. Most postmortems fail in the same three places.

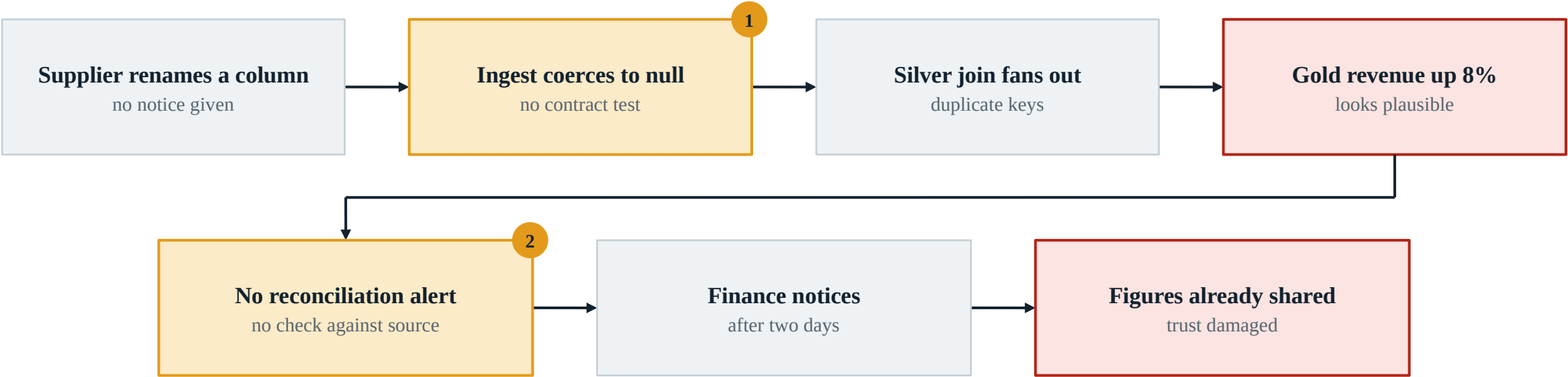
Section	What goes in it	Common failure
Summary	Impact, duration, detection and resolution in five lines	Buried under detail
Timeline	UTC timestamps of events, decisions and actions	Reconstructed from memory
Impact	Who was affected, and for how long data was wrong	"Minor" with no numbers
Contributing factors	Technical, process and signal gaps	Stops at "human error"
What went well	Detection, teamwork, and any luck worth naming	Skipped entirely
Action items	Owner, due date and how completion is verified	Vague or unowned
Follow-up	Dates to review whether the actions worked	Never revisited

PRACTICE

Review open action items weekly. Unfinished actions are the leading indicator of a repeat incident.

Tracing the causal chain

Illustrative composite incident. Every link is a place a control could have interrupted it.



1

PRACTICE

A contract test (Part 4) would have failed the ingest the moment the column changed.

2

PRACTICE

A reconciliation SLI (Part 6) would have flagged the 8% gap within hours.

FIVE WHYS

Ask why five times, but stop at the system, never at a person.

Playbooks for common data failures

Write these before you need them, then rehearse them.

Symptom	First checks	Containment	Durable fix
Data is late	Upstream status, orchestrator, quotas	Notify consumers; serve last good	Deadline alerts; supplier SLA
Duplicate rows	Retry logs, run ledger, key uniqueness	Dedupe the partition; pause retries	Idempotent writes; uniqueness tests
Schema changed	Contract diff, upstream release notes	Pin the previous version; quarantine	Contract tests; expand and contract
Numbers look wrong	Row counts, joins, recent deploys	Repoint to last good; label the data	Reconciliation; data diff in CI
Model degraded	Input drift, feature nulls, deploy log	Move the alias back	Shadow gates; drift alerts
Cost spike	Billing by tag, running jobs	Stop the job; cap concurrency	Budgets; unit-cost alerts

GAME DAYS

Once a quarter, rehearse one playbook with a synthetic failure. The gaps you find are cheap; the ones found live are not.

Measuring incident response

Illustrative targets for SEV1 and SEV2. Trend matters more than any single number.

< 10 min

Time to detect
incident start to first alert

< 60 min

Time to mitigate
detection to consumers protected

100%

Postmortems published
within five working days

< 10%

Repeat incidents
sharing a known cause

Metric	How to count it
Detect	Start of consumer impact to the first alert or report
Mitigate	Detection to the moment consumers can no longer act on bad data
Repeat rate	Incidents whose cause appeared in an earlier postmortem

Gate 6: incident readiness

What must exist before the first real incident.

Prepare

- ☐ Severity classes agreed with the business
- ☐ Roles and an on-call rota are named
- ☐ Playbooks exist for the top failure modes
- ☐ Pointer or rollback paths are rehearsed

Respond

- ☐ Declaration criteria are unambiguous
- ☐ A status template and channel are ready
- ☐ Containment options are pre-approved
- ☐ Consumers have one place to check status

Learn

- ☐ Blameless postmortem for every SEV1 and SEV2
- ☐ Actions have an owner, date and verification
- ☐ Open actions reviewed weekly
- ☐ Game days run at least quarterly

RULE

An incident with no change to the system will happen again. The action items are the product of the incident.

08

Cost review

Spend is a design constraint. Review it monthly, against unit costs, with an owner.

SLIDES 135–144 · DEPTH 5

IN THIS PART

- Where data platform money actually goes
- Tagging, allocation and showback
- Unit economics: cost per thing that matters
- Measuring unit cost from billing and ledger data
- Cost anomaly detection and its limits
- Levers, their risks and the monthly review
- Gate 7 checklist

Where data platform money goes

Six categories cover most of the bill. Each has a characteristic waste pattern.

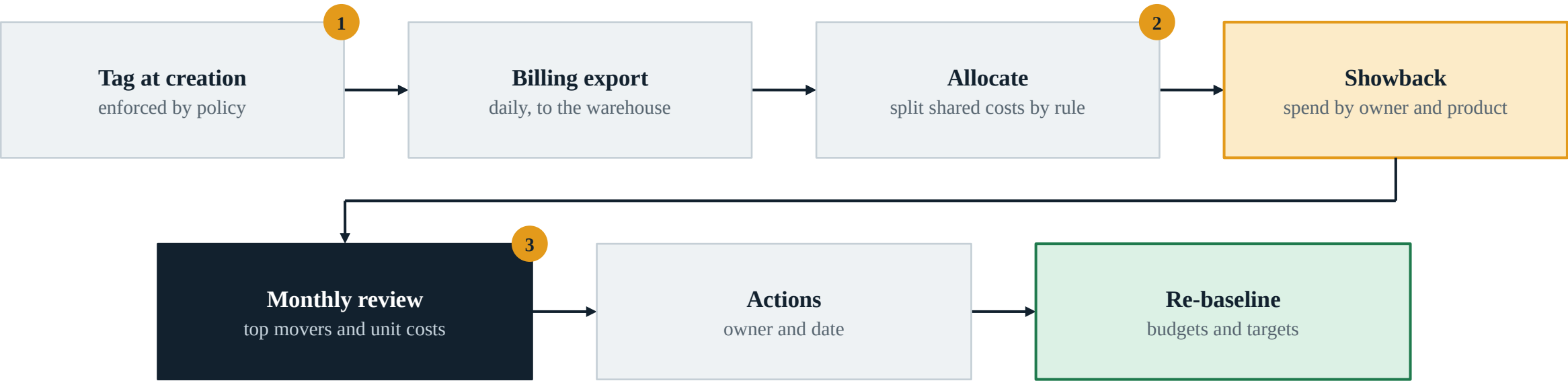
Category	Typical culprits	Levers
Storage	Unbounded raw and intermediate copies; no expiry	Lifecycle rules; retention policy; deduplication
Compute	Idle clusters, oversized jobs, retries, full refreshes	Auto-terminate; right-size; incremental builds
Scans	Unpruned queries, SELECT *, unpartitioned tables	Partitioning; clustering; query review
Data transfer	Cross-region traffic and egress; chatty pipelines	Co-locate; compress; cache
Managed services	Always-on endpoints; per-seat tools	Schedule off-hours; review licences
Experiments	Forgotten notebooks and GPU instances	TTL on dev environments; budgets per team

PRACTICE

Most surprise bills trace to just two things: a job that ran far more often than intended, and data that was never expired.

From tags to a monthly decision

Cost is only reviewable when every pound has an owner.



1

PRACTICE

Tags come from Terraform default_tags and the policy check in Part 5. Untagged spend is unowned spend.

2

NOTE

Shared platform costs need a stated split rule, for example by bytes scanned or jobs run.

3

THE POINT

The review compares unit cost with budget and explains every large mover.

Unit metrics: cost per thing that matters

Total spend rises with success. Unit cost tells you whether the platform is getting more efficient.

Unit metric	Numerator	Denominator	Watch for
Cost per 1,000 forecasts	Tagged platform spend	Forecast rows written	Retries inflating the denominator
Cost per TB processed	Compute and scan spend	TB scanned by jobs	Rewards scanning more; pair with a value metric
Cost per successful run	Job spend	Successful runs	Retries hiding inside "success"
Cost per model trained	Training compute	Registered models	Ignores experiments that never register
Storage per active TB	Storage spend	Actively queried TB	Cold data mislabelled as active

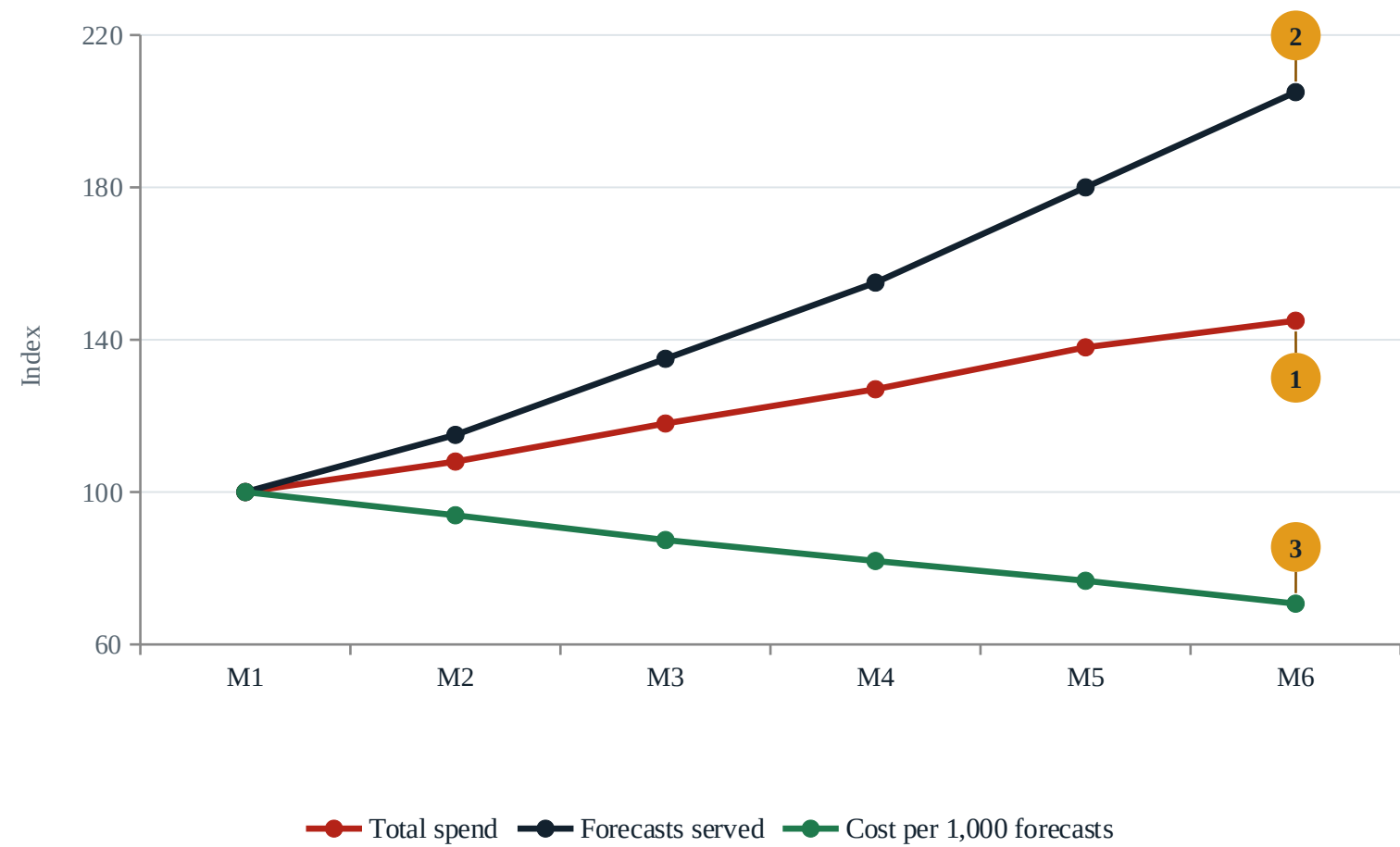
PITFALL

Every ratio can be gamed by moving its denominator. Track two or three together and review them for honesty.

Spend up 45%, unit cost down 29%

A rising bill is not automatically a problem. Compare it with what the platform delivered.

Figure 9. Monthly index, month 1 = 100 (illustrative)



1
PITFALL
Spend is up 45% over six months. A raw-spend alarm would fire.

2
NOTE
Volume more than doubled over the same period.

3
PRACTICE
Unit cost fell 29%. The platform is getting more efficient, and the review should say so.

Computing unit cost from billing and the run ledger

Tags supply the numerator. The ledger supplies the denominator.

```
SELECT date_trunc('month', u.usage_date) AS usage_month,  
       sum(u.cost_gbp) AS spend_gbp,  
       sum(u.cost_gbp) / (max(f.forecasts) / 1000.0) AS gbp_per_1k  
FROM billing.usage u  
JOIN (SELECT date_trunc('month', logical_date) AS m,  
          sum(rows_written) AS forecasts  
      FROM ops.run_ledger  
      WHERE job_name = 'batch_score'  
            AND status = 'success'  
      GROUP BY 1) f  
ON f.m = date_trunc('month', u.usage_date)  
WHERE u.tag_cost_center = 'forecasting'  
GROUP BY 1;
```

SQL

1

NUMERATOR

Tagged billing rows are the numerator. Untagged spend does not appear here, so track it separately.

1

2

2

LEDGER

The ledger from Part 2 is now a cost instrument, not just an audit record.

3

3

DENOMINATOR

Count only successful runs, so retries do not inflate the denominator.

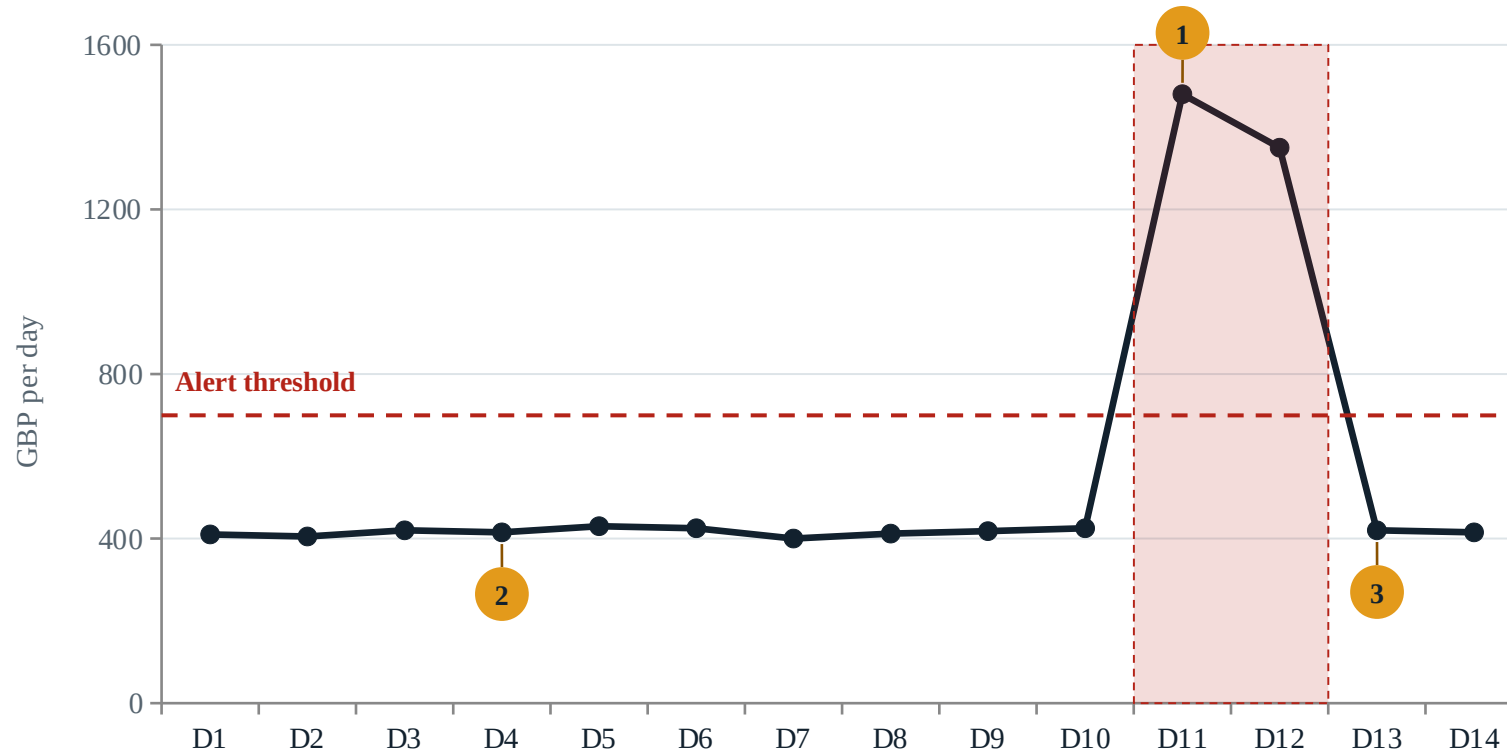
NOTE

Column names are illustrative. Billing exports differ by cloud, and tag columns need enabling for cost allocation.

Cost anomalies: detect them, and know the delay

A runaway backfill triples the daily bill. How fast do you find out?

Figure 10. Daily platform spend, GBP, over 14 days (illustrative)



1

PITFALL

Spend more than triples. A backfill ran on the shared pool with no concurrency cap.

2

NOTE

A baseline near 415 makes a 700 threshold easy to justify and tune.

3

THE DELAY

Billing data lags by hours or a day, so the alert fires after the damage. Job-level estimates react sooner.

Cost levers and what each one puts at risk

Every saving has a failure mode. Name the guard before pulling the lever.

Lever	Effect	Risk	Guard
Storage tiering and expiry	Steady cut on cold data	Deleting data you must keep	Retention policy first; rules in code
Partition pruning and clustering	Fewer bytes scanned	Query rewrites change results	Data diff in CI (Part 4)
Auto-terminate and right-size	Idle spend removed	Undersizing causes failures	Load test; duration SLO
Spot or preemptible compute	Discount on batch work	Interruptions mid-run	Idempotent, resumable jobs
Incremental instead of full refresh	Large cut on big tables	Drift from full-refresh truth	Periodic rebuild plus diff
Cache and materialise hot queries	Repeated scans avoided	Stale results	Freshness contract on the cache

RULE

Never cut a cost that protects an SLO, an audit trail or a rollback path without an explicit decision by the owner.

The monthly cost review

One hour, the same agenda every month, with owners in the room.

1	Prepare	Assemble showback, unit costs and budget position beforehand.	OUTPUT <i>One shared dashboard; no live queries</i>
2	Top movers	List the biggest increases and decreases, by owner.	OUTPUT <i>Ranked table of changes</i>
3	Explain	Tie each change to volume, a release, a backfill or waste.	OUTPUT <i>A stated cause for every large mover</i>
4	Decide	Act, accept, or re-budget. Say which, out loud.	OUTPUT <i>A decision recorded per item</i>
5	Assign	Give each action a single owner and a date.	OUTPUT <i>Action list with owners and dates</i>
6	Record	Update the forecast, targets and the changelog.	OUTPUT <i>Notes published within a day</i>

PRACTICE

Review last month's actions first. A cost review that never checks its own follow-through decays into a status meeting.

Gate 7: cost readiness

Evidence that spend is owned, measured and reviewed.

Allocate

- ☐ Tags enforced by policy at creation
- ☐ Billing exported to the warehouse daily
- ☐ Shared cost split rules are documented
- ☐ Showback reaches every owner monthly

Measure

- ☐ A headline unit cost per data product
- ☐ Denominators counted from successful runs
- ☐ Budgets with alerts at set thresholds
- ☐ Anomaly detection plus per-job estimates

Govern

- ☐ Backfill plans include a cost estimate
- ☐ Monthly review with recorded decisions
- ☐ Lever risks have named guards
- ☐ Dev environments expire by default

RULE

If nobody can say who owns a line on the bill, nobody will reduce it.

09

Capstone

Bring the seven gates together, choose what to do first, and start.

SLIDES 145–150 · DEPTH 5

IN THIS PART

- The readiness review scorecard
- Prioritising by impact and effort
- A 90-day plan
- Anti-patterns in hardening programmes
- Closing summary

The production readiness review

Score each gate 0 to 3 for every data product. The pattern of gaps is your plan.

Gate	Core question	Minimum evidence
1 · Idempotency	Can any job run twice safely?	Run-twice and crash tests in CI
2 · Backfills	Can history be rebuilt without harm?	One-page plan, shadow table, ledger and diff
3 · CI/CD	Do changes ship through gates?	Contracts, data diffs and a rollback per artifact
4 · Terraform	Is infrastructure reviewed code?	Split state, saved plans, drift detection
5 · Monitoring	Will we know before consumers do?	SLOs, burn or deadline alerts, heartbeat
6 · Incidents	Can we contain and learn quickly?	Roles, playbooks, blameless postmortems
7 · Cost	Is spend owned and understood?	Tags, unit cost and a monthly review

SCORING

0 = absent, 1 = informal, 2 = in place and used, 3 = automated and reviewed. Illustrative scale; adapt to your organisation.

Choosing what to do first

Illustrative placements for the running example. Yours will differ, and that is the point of scoring.

Impact: low ↑ high

High impact, low effort: do first

- Run-twice tests on the top pipelines
- Freshness and deadline alerts on gold
- Owner and cost-centre tags everywhere

High impact, high effort: plan it

- Contracts across producing teams
- Terraform migration and state split
- Lineage-aware alert routing

Low impact, low effort: fill gaps

- Runbook stubs for every alert
- Retire unused dashboards
- Expire dev environments by default

Low impact, high effort: avoid

- Bespoke tooling rewrites
- Perfect coverage before any gate ships
- Custom anomaly models for minor tables

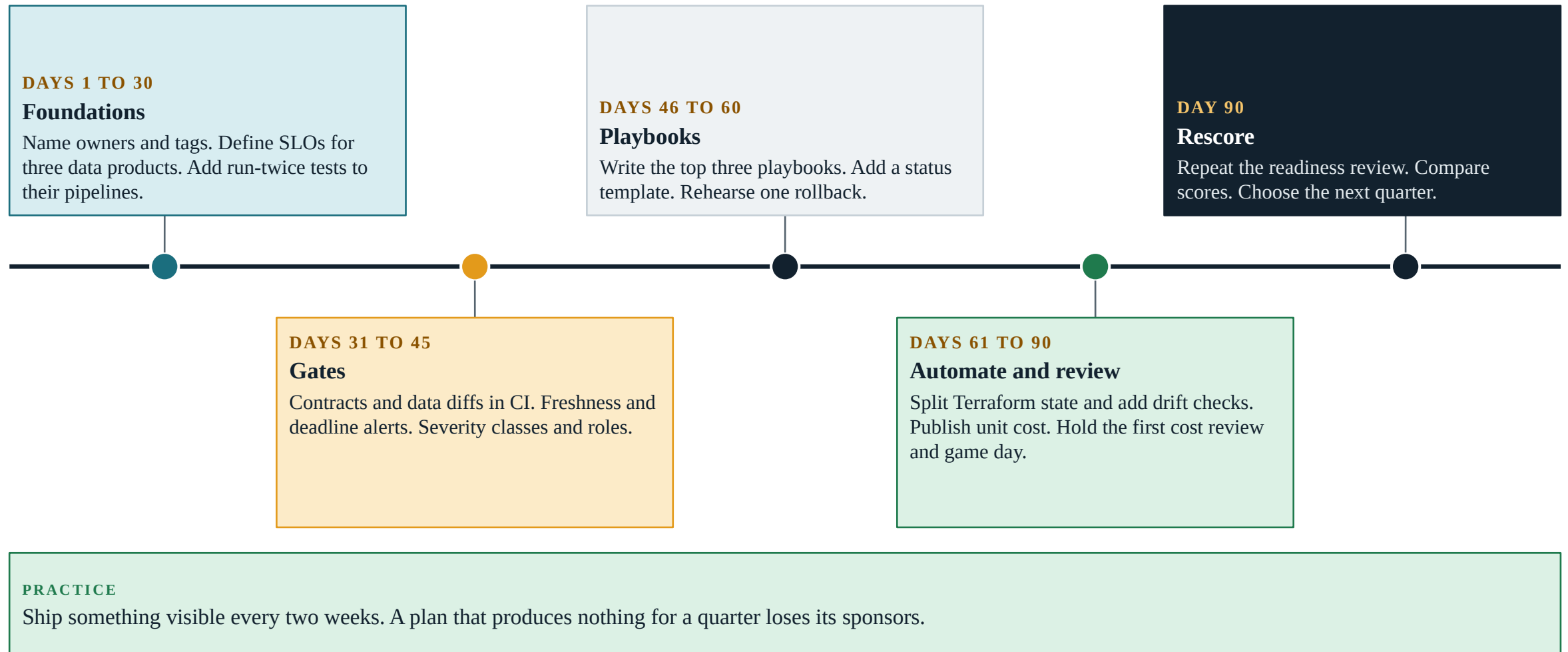
Effort: low → high

RULE

Start with the pipelines feeding your SLOs. Ratchet: gate all new work now, and improve old work as you touch it.

A 90-day plan

Foundations, then gates, then automation, then review. Illustrative sequencing.



Anti-patterns in hardening programmes

The technical work is usually the easy part. These are the organisational traps.

Tempting		Better
Harden everything at once	➔	Start with the pipelines that feed your SLOs
Buy a platform to fix the culture	➔	Adopt the patterns first; choose tools second
Alert on everything, to be safe	➔	Alert on SLO burn and deadlines; ticket the rest
Wait for perfect coverage before shipping gates	➔	Gate new work now; ratchet older work up
Treat hardening as a one-off project	➔	Run the loop: measure, respond, learn, re-baseline
PITFALL Every pattern in this deck is a trade-off. Adopt the smallest version that removes a real risk you have actually seen.		

THE TAKEAWAY

Harden for the second run, the tenth rebuild and the 03:00 page. Then let the system tell you what to harden next.

Idempotent by default

Backfills treated as releases

CI/CD with data diffs and contracts

Infrastructure as reviewed, drift-checked code

Alerts driven by SLO burn and deadlines

Blameless learning and unit-cost review